Python Programming

COURSE EXERCISES

Exercise 1 : Python Intro 3

Exercise 2 : Variables and Data Types 9

Exercise 3 : Data Structures 16

Exercise 4 : Control Flow 24

Exercise 5 : Functions 37

Exercise 6 : Modules 48

Exercise 7 : Comprehensions 55

Exercise 8 : Exception Handling 65

Exercise 9 : Python IO 72

Exercise 10 : Database Access 84

Exercise 11 : Python Classes 98

APPENDIX

Exercise 12 : Type Hinting 110

Exercise 13 : Python Libraries 118

Exercise 1 : Python Intro

Before you can write a game you need somewhere to write it. This exercise installs Python, the `uv` project manager, the `ruff` linter and an editor that understands Python, and creates the project you build a Wordle game in for the rest of the course.

There is no starter for this exercise : it is where your own `wordle` project begins. From Exercise 2 on, every exercise also has a starter folder for when you fall behind.

Every step in this exercise gives you the commands : the point is to see each tool work once.

Step 1 : Install uv

`uv` is the project manager the course uses. It can install Python itself, so it is the only thing you install by hand. On Windows, run this in PowerShell :

```
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

On macOS or Linux, run this in a terminal :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close and reopen the terminal afterwards so that `uv` is on your `PATH`, then confirm that it works :

```
uv --version
```

The documentation is at `https://docs.astral.sh/uv/`.

Step 2 : Install Python

Let `uv` fetch the interpreter the course uses :

```
uv python install 3.14
```

This needs no administrator rights and leaves any Python already on the machine alone. Confirm it with `uv python list`, which shows every interpreter `uv` can see.

Step 3 : Install an editor

Install Visual Studio Code from `https://code.visualstudio.com`, then add two extensions from the Extensions panel, the square icon on the left :

- `Python`, published by Microsoft
- `Ruff`, published by Astral

Any editor works, but the course demonstrates VS Code and the slides assume it. The slide *Editor Setup* lists what the two extensions do for you.

Step 4 : Create the wordle project

You write your own code this week in a project of your own, the same way you would at work. From the folder where you keep your work, run :

```
uv init wordle
cd wordle
```

`uv init` writes a `pyproject.toml`, a `.python-version`, a `README.md` and a placeholder `main.py`, and makes the folder a git repository; the slide *Creating a Project* explains each of them. Run the placeholder :

```
uv run python main.py
```

The first run creates the project's own environment in `.venv`, then prints :

```
Hello from wordle!
```

Open the `wordle` folder in VS Code. Then open the Command Palette, run `Python: Select Interpreter`, and choose the interpreter in `.venv`. The editor now uses the same Python as `uv run`.

Step 5 : See the linter work

A linter reads code without running it and reports what looks wrong. Add `ruff` to the project as a development tool, one the project needs while you write it but not when it runs :

```
uv add --dev ruff
```

`uv` records it in `pyproject.toml` under `[dependency-groups]`, so anybody who works on the project gets the same tool. The test runner `pytest` arrives the same way in the libraries appendix, in Exercise 13. Run `ruff` over the project :

```
uv run ruff check .
```

There is nothing wrong yet, so it reports :

```
All checks passed!
```

Now write something with a mistake in it. Create `hello.py` in the project :

```
import json


def greet(name: str) -> str:
    return f"Hello, {name}"


print(greet("Python"))
```

Run it with `uv run python hello.py`. It prints `Hello, Python`, so Python itself has no complaint. Run `uv run ruff check .` again. This time `ruff` reports the import nobody uses :

```
F401 [*] `json` imported but unused
 --> hello.py:1:8
  |
1 | import json
  |        ^^^^
help: Remove unused import: `json`
  |
  - import json
1 |
  |

Found 1 error.
[*] 1 fixable with `--fix`.
```

`F401` is the rule, and the arrow points at the file, line and column. VS Code underlines the same line, because the Ruff extension runs the same checks as you type. The `[*]` means the fix is safe to make automatically, so let `ruff` make it :

```
uv run ruff check . --fix
```

It reports `Found 1 error (1 fixed, 0 remaining).`, and the import has gone from `hello.py`. The slide *Linting with ruff* has the other commands. You can delete `hello.py` now; the game does not need it.

Step 6 : The word list

A word game needs words. Copy `words.txt` from `exercises/handouts/01-environment-configuration/` into the `wordle` folder, next to `pyproject.toml`.

Rename `main.py` to `wordle.py`, which is the name the game has from here on, and replace its contents with :

```
from pathlib import Path


def main() -> None:
    """Print the number of characters in the word list."""
    words = Path("words.txt").read_text(encoding="utf-8")
    print(f"The word list contains {len(words)} characters.")


if __name__ == "__main__":
    main()
```

`Path("words.txt").read_text(...)` reads the whole file into one string, and `len` counts its characters. The rest of the file is the same shape as the `main.py` that `uv init` wrote; module 05 explains `def` and module 06 explains the last two lines. Run it :

```
uv run python wordle.py
```

It prints :

```
The word list contains 5610 characters.
```

The file holds one five-letter word per line, so every word takes six characters including its line break. How many words are in the list? Exercise 3 counts them properly.

If time permits

The course folder has a file that is written badly on purpose, `demos/01-pythonintro/03_ruff_findings.py`. Copy it into your `wordle` project and run `uv run ruff check 03_ruff_findings.py`. Read every finding, and decide why `ruff` objects before you read the comment in the file that explains it.

Then copy `ruff.toml` from the course folder into the project as well, and run the same command again. The course's configuration switches on more rules than the default set, so `ruff` now finds more. Which rules did it add, and would you want them in your own projects? Delete both files when you are done.

Exercise 2 : Variables and Data Types

A Wordle game has to compare a guess with the secret word, letter by letter, before it can tell you anything. In this exercise you do that for one guess against one known word, using nothing but strings and f-strings.

Continue in your own `wordle` project, or start from `exercises/starters/02-variables-and-data-types/`, which is the finished Exercise 1. Run the program after every step with :

```
uv run python wordle.py
```

Some steps give you the code, some a skeleton to complete, and some only a description of what to write. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : A secret and a guess

Replace the contents of `wordle.py` with a fixed secret word and a guess typed by the player :

```
secret = "crane"
guess = input("Your guess: ").strip().lower()

print(f"{' WORDLE ':*^28}")
print(f"Secret: {secret.upper()}    Guess: {guess.upper()}")
```

`input` always returns a string. `.strip()` removes the spaces a player types by accident around the word, and `.lower()` makes `Slate` and `slate` the same guess. Each method returns a new string, so they can be chained.

In the banner, `:*^28` is a format specification : centre the text (`^`) in 28 characters, and fill the space on both sides with `*`. The slides *f-Strings* and *Format Specifications* show the other options. Run the program and type `Slate` with the

spaces :

```
Your guess:   Slate
********** WORDLE **********
Secret: CRANE    Guess: SLATE
```

Step 2 : Letter by letter

Until Exercise 4 adds the `if` statement, the program cannot check the length of the guess, so type a guess of exactly five letters. A shorter guess stops this step with an `IndexError`, because `guess[4]` does not exist.

For each of the five positions, the game needs the answer to two questions : is the letter of the guess the same as the letter of the secret in that place, and is the letter of the guess anywhere in the secret? Print the answers as a small table, one row per position.

The header and the shape of the first row are given. Fill in the two questions, then add the rows for positions 1 to 4 :

```
print(f"{'Position':^10}{'Same place':^14}{'In secret':^12}")
print(f"{0:^10}{...!s:^14}{...!s:^12}")
```

The widths 10, 14 and 12 keep every column under its heading. The `!s` before each colon turns the answer into a string first : with a width and without `!s`, a `bool` is formatted as the number it also is, and the table shows `1` and `0` instead of `True` and `False`.

> ***Hint :*** *The slide String Operations takes one character out of a string with `str1[0]`, and Logical, Membership, Identity Operators has the `in` operator.*

> ***Hint :*** *`guess[0]` is the first letter of the guess. Comparing two letters with `==` gives `True` or `False`, and so does the `in` operator : `"a" in "crane"` is `True`.*

Expected output :

```
Your guess: react
********** WORDLE **********
Secret: CRANE    Guess: REACT
 Position   Same place   In secret
    0         False         True
    1         False         True
    2          True         True
    3         False         True
    4         False        False
```

Solution on next page ↓

Solution

```
print(f"{'Position':^10}{'Same place':^14}{'In secret':^12}")
print(f"{0:^10}{guess[0] == secret[0]!s:^14}{guess[0] in
secret!s:^12}")
print(f"{1:^10}{guess[1] == secret[1]!s:^14}{guess[1] in
secret!s:^12}")
print(f"{2:^10}{guess[2] == secret[2]!s:^14}{guess[2] in
secret!s:^12}")
print(f"{3:^10}{guess[3] == secret[3]!s:^14}{guess[3] in
secret!s:^12}")
print(f"{4:^10}{guess[4] == secret[4]!s:^14}{guess[4] in
secret!s:^12}")
```

Five lines that differ only in a number are a loop waiting to be written. Exercise 4 writes it.

Step 3 : Length, slices and counts

Three more questions about the guess and the secret, each answered in one line :

```
print(f"Five letters: {...}")
print(f"First two: {...}, last two: {...}")
print(f"The secret contains {...} letter e characters.")
```

The first is a check the game will need on every guess : does it have five letters? It only prints `True` or `False` for now. It does not protect the table of Step 2, which indexes the guess before this line runs, so keep typing five letters. The second takes the guess apart with slices. The third counts how often `e` occurs in the secret.

> ***Hint :*** *The slide String Operations shows slices, including ones that count from the end, and Built-in String Methods has* `len` *and the string methods.*

> ***Hint :*** `len` *gives the length of a string, and comparing it with* `==` *gives* `True` *or* `False` *. A slice* `word[start:stop]` *can leave out either end, and a negative index counts from the end :* `word[-1]` *is the last letter.*

> ***Hint :*** *Strings have a method that counts how often a piece of text occurs in them, and it is named after what it does. You call it on the secret, with the letter as its argument.*

Solution on next page ↓

Solution

```
print(f"Five letters: {len(guess) == 5}")
print(f"First two: {guess[:2]}, last two: {guess[-2:]}")
print(f"The secret contains {secret.count('e')} letter e characters.")
```

Inside this double-quoted f-string, single quotes make the inner string literal easy to distinguish : `'e'`. Python 3.14 also allows double quotes inside the expression.

Step 4 : Tabs and raw strings

A backslash in a string starts an escape sequence : `\t` is a tab and `\n` a line break. Add a second header for the table, laid out with tabs instead of widths, and a line that shows a backslash as it is written :

```
print("Position\tSame place\tIn secret")
print(r"The escape sequence \t inserts a tab.")
```

The `r` in front of the second string makes it a raw string, in which a backslash is an ordinary character. Remove the `r`, run the program, and see what happens to the line. Then put it back. The slides *Escape Characters* and *Raw and Unicode Strings* list the other sequences.

Run the complete program once more and compare :

Expected output :

```
Your guess: react
********** WORDLE **********
Secret: CRANE    Guess: REACT
 Position   Same place   In secret
    0         False         True
    1         False         True
    2          True         True
    3         False         True
    4         False        False
Five letters: True
First two: re, last two: ct
The secret contains 1 letter e characters.
Position    Same place  In secret
The escape sequence \t inserts a tab.
```

The complete file is in `solutions/02-variables-and-data-types/wordle.py`.

If time permits

A game has a number of guesses. Ask the player how many they want, convert the answer with `int()`, and print it back in a sentence. Then run the program again and answer `six`. Read the last line of the error that Python prints : which exception is it, and what does its message tell you about what `int()` accepts? The slide *Type Conversions* shows the conversion functions. There is no reference solution for this step.

Exercise 3 : Data Structures

So far the game knows one word, `crane`, and forgets every guess the moment it has printed it. A real game picks its secret from a list of words, and keeps track of what each guess has ruled out. In this exercise you load the word list and use lists, sets, tuples and dictionaries to answer questions about a guess.

Continue in your own `wordle` project, or start from `exercises/starters/03-data-structures/`, which is the finished Exercise 2 with the word list. This exercise starts a new script in `wordle.py`; Exercise 2's table is not needed again. Run the program after every step with :

```
uv run python wordle.py
```

Some steps give you the code, some a skeleton to complete, and some only a description of what to write. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : The word list

Replace the contents of `wordle.py` with :

```
import random
from pathlib import Path

words = Path("words.txt").read_text(encoding="utf-8").split()
guess = input("Your guess: ").strip().lower()
random.seed(0)
secret = random.choice(words)

print(f"There are {len(words)} words. The first ten are {words[:10]}.")
print(f"Crane is in the word list: {'crane' in words}.")
print(f"Example secret: {secret.upper()}.")
```

The line that reads the file is an incantation for now : it reads `words.txt` into one string, and `.split()` cuts that string into a list wherever there is white space, which in this file means at every line break. Exercise 9 takes it apart.

`words` is a list of strings, so `len`, slicing and `in` work on it just as they did on a single string. `random.choice` picks one item of a list at random. `random.seed(0)` makes the random choices the same on every run, which is why everybody gets the same example secret below. Remove that line and run the program twice to see the secret change, then put it back so your output matches the rest of this exercise.

Run it and type `crane` :

```
Your guess: crane
There are 935 words. The first ten are ['abbey', 'abide', 'abled',
'abort', 'about', 'above', 'abuse', 'acorn', 'actor', 'acute'].
Crane is in the word list: True.
Example secret: TRUST.
```

Step 2 : What a guess rules out

`set("crane")` is the set of letters in a word : each letter once, in no particular order. Two sets can be combined with operators, which is exactly what the game needs to describe a guess. Work out three sets of letters, each sorted into a list for printing :

- the letters the guess and the secret have in common
- the letters of the guess that are not in the secret, which the guess has ruled out
- the letters of the alphabet that the guess has not tried

```
import string  # at the top of the file, with the other imports

common = sorted(...)
ruled_out_letters = sorted(...)
not_tried = sorted(...)
print(f"Letters in common: {common}")
print(f"Letters ruled out: {ruled_out_letters}")
print(f"Letters not tried: {not_tried}")
```

`string.ascii_lowercase` is the string `"abcdefghijklmnopqrstuvwxyz"`, the alphabet the third set starts from.

> ***Hint :*** *The slide Set Operations shows the operators :* `&` *gives the items that are in both sets, and* `-` *gives the items of the first set that are not in the second.*

> ***Hint :*** *Make a set of* `string.ascii_lowercase` *the same way as of a word.* `sorted` *accepts a set and returns a list, which is why the result prints in alphabetical order.*

Expected output :

```
Letters in common: ['r']
Letters ruled out: ['a', 'c', 'e', 'n']
Letters not tried: ['b', 'd', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
'o', 'p', 'q', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```

Solution on next page ↓

Solution

```
common = sorted(set(guess) & set(secret))
ruled_out_letters = sorted(set(guess) - set(secret))
not_tried = sorted(set(string.ascii_lowercase) - set(guess))
print(f"Letters in common: {common}")
print(f"Letters ruled out: {ruled_out_letters}")
print(f"Letters not tried: {not_tried}")
```

Step 3 : A history of guesses

A game has more than one guess, and they come in order. Keep them in a list called `history` : start it empty, then append a first guess, `"slate"` , and the guess the player typed.

Then record a guess together with what it ruled out as a pair, the tuple `(guess, ruled_out_letters)` , in a variable called `ruled_out` . Print both :

```
history = ...
...
ruled_out = ...
print(f"History: {history}")
print(f"Guess and ruled-out letters: {ruled_out}")
```

A tuple cannot be changed once it is made. See that for yourself : add the line `ruled_out[0] = "wrong"` below the tuple, run the program, and read the last line of the error. It is a `TypeError` that says a `tuple` does not support item assignment. Then turn the line into a comment, so the program runs again and the reader can still see what you tried.

> ***Hint :*** *`demo01 · lists` starts with an empty list, the slide List Functions and Methods has `append`, and Tuples shows how a tuple is written.*

> ***Hint :*** *`[]` is an empty list, and `append` adds one item at the end. A tuple is written as values separated by commas, usually in brackets.*

Expected output :

```
History: ['slate', 'crane']
Guess and ruled-out letters: ('crane', ['a', 'c', 'e', 'n'])
```

Solution on next page ↓

Solution

```
history = []
history.append("slate")
history.append(guess)
ruled_out = (guess, ruled_out_letters)
# Uncomment this line to see why tuple items cannot be reassigned.
# ruled_out[0] = "wrong"
print(f"History: {history}")
print(f"Guess and ruled-out letters: {ruled_out}")
```

The tuple itself cannot change, but the list inside it still can : `ruled_out[1].append("z")` is allowed. A tuple fixes which objects it holds, not what happens inside them.

Step 4 : Statistics

A game keeps statistics : how many games were played and how many were won. A dictionary holds them by name. Create `stats` with the keys `"played"` and `"won"`, both starting at 0. Then record this game : one more played, and one more won if the guess was the secret.

Print the dictionary, and also the current streak, which `stats` does not have yet :

```
stats = ...
...
print(f"Stats: {stats}; streak: {stats.get('streak', 0)}")
```

`stats["streak"]` would stop the program with a `KeyError`, because the key does not exist. `get` returns the second argument instead. The slide *Dictionary Functions and Methods* lists `get`.

> ***Hint :*** *The slide Dictionaries shows a dictionary written out with its keys and values, and Accessing Values in Dictionary changes the value under a key.*

> ***Hint :*** `stats["played"] += 1` *adds one to the value stored under* `"played"`.

> ***Hint :*** *There is no* `if` *yet, and you do not need one.* `guess == secret` *is* `True` *or* `False`*, and as the slide Type Conversions says, a Boolean behaves like 0 and 1 :* `int(True)` *is 1 while* `int(False)` *is 0.*

Expected output :

```
Your guess: crane
There are 935 words. The first ten are ['abbey', 'abide', 'abled',
'abort', 'about', 'above', 'abuse', 'acorn', 'actor', 'acute'].
Crane is in the word list: True.
Example secret: TRUST.
Letters in common: ['r']
Letters ruled out: ['a', 'c', 'e', 'n']
Letters not tried: ['b', 'd', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
'o', 'p', 'q', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
History: ['slate', 'crane']
Guess and ruled-out letters: ('crane', ['a', 'c', 'e', 'n'])
Stats: {'played': 1, 'won': 0}; streak: 0
```

Solution on next page ↓

Solution

```
stats = {"played": 0, "won": 0}
stats["played"] += 1
stats["won"] += int(guess == secret)
print(f"Stats: {stats}; streak: {stats.get('streak', 0)}")
```

The complete file is in `solutions/03-data-structures/wordle.py`.

If time permits

Two more questions about the word list :

- Print the first ten words of the list sorted in reverse order, and then the first ten sorted by their last letter. `sorted` takes a `key`, a small function that says what to sort by : `sorted(words, key=lambda word: word[-1])`. Module 05 explains `lambda`.
- How many words start with each vowel? Without a loop this takes a trick : join the words into one long string with a space in front of every word, then `count` how often `" a"`, `" e"` and so on occur in it. Check one of your answers by looking at the word list.

There is no reference solution for this step.

Exercise 4 : Control Flow

Everything so far answers questions about a single guess, and then the program ends. With loops and conditions you can finally write the game itself : six guesses, a row of marks after each one, and a win or a loss at the end. At the end you have a game you can play.

Continue in your own `wordle` project, or start from `exercises/starters/04-control-flow/`, which is the finished Exercise 3. Run the game after every step with :

```
uv run python wordle.py
```

Some steps give you the code, some a skeleton to complete, and one only the rules and three games to check it with. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : Six guesses

Exercise 3's script explored the data; this exercise turns it into a game. Replace the contents of `wordle.py` with :

```
import os
import random
from pathlib import Path

words = Path("words.txt").read_text(encoding="utf-8").split()
secret = os.environ.get("WORDLE_SECRET", random.choice(words))
max_guesses = 6

print("Guess the five-letter word.")

guess_count = 0
while guess_count < max_guesses:
    guess = input(f"Guess {guess_count + 1}/{max_guesses} :
").strip().lower()
    guess_count += 1
    print(guess.upper())

print(f"The word was {secret.upper()}.")
```

The `while` loop runs its indented block again and again for as long as its condition is true. Every pass reads a guess and adds one to `guess_count`, so after six passes the condition is false and the program carries on below the loop. The slide *while Loop* shows the shape.

A random secret makes a game you cannot check, so the secret can be fixed from outside the program. `os.environ` holds the environment variables of the process, and `get` returns the second argument when `WORDLE_SECRET` is not set. In PowerShell, set it for the rest of the session :

```
$env:WORDLE_SECRET = "crane"
uv run python wordle.py
```

In bash, set it for a single run :

```
WORDLE_SECRET=crane uv run python wordle.py
```

The outputs in this exercise all use `crane`. In PowerShell, `Remove-Item Env:WORDLE_SECRET` brings the random secret back. Play once : the game takes any six answers, even ones that are not words, and does not yet tell you anything about them.

```
Guess the five-letter word.
Guess 1/6 : Slate
SLATE
Guess 2/6 : hello
HELLO
```

Step 2 : Rejecting a guess

A guess of the wrong length, or one that is not in the word list, should not cost the player a turn. Before `guess_count` goes up, check the guess, and when it is not acceptable print a message and go straight back to the top of the loop for another one :

```
    if ...:
        print("That is not a five-letter word I know.")
        continue

    guess_count += 1
```

`continue` ends the current pass of the loop and starts the next one, so nothing below it runs for a rejected guess. The slide *break and continue Statements* shows it next to `break`, which you need in Step 5.

> ***Hint :*** *The slide if Statement shows the shape of the check. Logical, Membership, Identity Operators, in module 02, has* `or` *and* `not in` *.*

> ***Hint :*** *There are two reasons to reject a guess, and either one is enough, so join them with* `or` *.* `len(guess) != 5` *is the first. For the second,* `not in` *is the opposite of* `in` *.*

Expected output :

```
Guess the five-letter word.
Guess 1/6 : slat
That is not a five-letter word I know.
Guess 1/6 : abcde
That is not a five-letter word I know.
Guess 1/6 : slate
SLATE
```

Solution on next page ↓

Solution

```
    if len(guess) != 5 or guess not in words:
        print("That is not a five-letter word I know.")
        continue

    guess_count += 1
```

The check comes before `guess_count += 1`. Put it after, and a rejected guess still uses up a turn.

Step 3 : A row of marks

Now the game should say something about each letter. Replace `print(guess.upper())` with a row that marks every letter :

- `[A]` : the letter is in the right place
- `(A)` : the letter is in the word, somewhere else
- `A` : the letter is not in the word

Build the row one letter at a time, starting from an empty string :

```
    # [A] is in the right place, (A) is in the word elsewhere, A is
not in it.
    row = ""
    for position, letter in enumerate(guess):
        if ...:
            row += f"[{letter.upper()}]"
        elif ...:
            row += ...
        else:
            row += ...
    print(row)
```

`enumerate` gives each letter together with its position, 0 to 4, which is what you need to compare it with the letter in the same place of the secret.

> ***Hint :*** *The slide Iterating with enumerate shows the loop, and elif Statement shows the chain of conditions.*

> ***Hint :*** *The first condition is the "same place" question of Exercise 2, asked with* `letter` *and* `position` *:* `letter == secret[position]` *. The second is the "in secret" question.*

> ***Hint :*** *Each mark is three characters wide, so the rows line up under each other. The last case is a space, the letter, and a space :* `f" {letter.upper()} "` *.*

Expected output :

```
Guess the five-letter word.
Guess 1/6 : slate
 S  L [A] T [E]
Guess 2/6 : error
(E)[R](R) O (R)
Guess 3/6 : crane
[C][R][A][N][E]
Guess 4/6 :
```

Solution on next page ↓

Solution

```
    # [A] is in the right place, (A) is in the word elsewhere, A is
not in it.
    row = ""
    for position, letter in enumerate(guess):
        if letter == secret[position]:
            row += f"[{letter.upper()}]"
        elif letter in secret:
            row += f"({letter.upper()})"
        else:
            row += f" {letter.upper()} "
    print(row)
```

The game still asks for a fourth guess after `crane`, because nothing tells the loop that the word has been found. Step 5 does that.

Step 4 : Commands

Two words are not guesses but commands : `quit` gives up, and `letters` shows the letters tried so far, in alphabetical order. A set is the natural place to keep the letters tried, because each letter only needs to be in it once.

Create the set before the loop, and add the letters of every accepted guess to it. Change the welcome line, so the player knows about the commands :

```
max_guesses = 6
used_letters: set[str] = set()

print("Guess the five-letter word. Type 'quit' to give up, 'letters'
for the letters tried.")
```

```
    guess_count += 1
    used_letters.update(guess)
```

Then check for the commands straight after reading the guess, before the check of Step 2 :

```
    match guess:
        case "quit":
            ...
        case "letters":
            ...
```

For now, `quit` only has to leave the loop; the line below the loop then prints the word.

> ***Hint :*** *The slide match Statement shows the syntax, and break and continue Statements shows* `break` *. Built-in String Methods has* `sep.join(words)` *, and Set Operations turns a set into a sorted list with* `sorted()` *.*

> ***Hint :*** `break` *leaves the loop at once. For* `letters` *,* `sorted(used_letters)` *gives a list of the letters in alphabetical order, and* `" ".join(...)` *turns that list into one string with a space between the letters :* `print("Tried :", " ".join(sorted(used_letters)))` *.*

> ***Hint :*** *Without a* `continue` *after printing the letters, the program goes on to the check of Step 2, and* `letters` *is not a five-letter word.*

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : slate
 S  L [A] T [E]
Guess 2/6 : letters
Tried : a e l s t
Guess 2/6 : error
(E)[R](R) O (R)
Guess 3/6 : quit
The word was CRANE.
```

Solution on next page ↓

Solution

```
    match guess:
        case "quit":
            break
        case "letters":
            print("Tried :", " ".join(sorted(used_letters)))
            continue
```

`set()` is an empty set; `{}` would be an empty dictionary. `update` adds every item of what it is given, so `used_letters.update("slate")` adds five letters.

Step 5 : Winning and losing

The game has to end properly. These are the rules :

- when the guess is the secret, print `Got it in N!`, with the number of guesses it took, and end the game
- when the six guesses are used up without finding the word, print `Out of guesses. The word was CRANE.`
- when the player types `quit`, print `The word was CRANE.` and end the game
- in every case, the program prints exactly one of those lines

The line below the loop that prints the word goes. Win with `break`, and lose with an `else:` on the `while` loop. Check it with three games against `crane` : one won, one given up and one lost. Play all three and compare with the expected output.

```
Game 1 : slate, error, crane
Game 2 : slate, quit
Game 3 : slate, error, house, ghost, world, money
```

> ***Hint :*** *The slide Loop with else Combination shows it : a loop can have an `else:` block, at the same indentation as the `while` . It runs when the loop ends because its condition became false, and it does not run when the loop is left with `break` . That is exactly the difference between running out of guesses and finding the word.*

> ***Hint :*** `quit` *leaves the loop with* `break` *as well, so it skips the* `else:` *too. That is why the* `quit` *case now prints the word itself.*

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : slate
 S  L [A] T [E]
Guess 2/6 : error
(E)[R](R) O (R)
Guess 3/6 : crane
[C][R][A][N][E]
Got it in 3!
```

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : slate
 S  L [A] T [E]
Guess 2/6 : quit
The word was CRANE.
```

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : slate
 S  L [A] T [E]
Guess 2/6 : error
(E)[R](R) O (R)
Guess 3/6 : house
 H  O  U  S [E]
Guess 4/6 : ghost
 G  H  O  S  T
Guess 5/6 : world
 W  O (R) L  D
Guess 6/6 : money
 M  O (N)(E) Y
Out of guesses. The word was CRANE.
```

Solution on next page ↓

Solution

```
import os
import random
from pathlib import Path

words = Path("words.txt").read_text(encoding="utf-8").split()
secret = os.environ.get("WORDLE_SECRET", random.choice(words))
max_guesses = 6
used_letters: set[str] = set()

print("Guess the five-letter word. Type 'quit' to give up, 'letters'
for the letters tried.")

guess_count = 0
while guess_count < max_guesses:
    guess = input(f"Guess {guess_count + 1}/{max_guesses} :
").strip().lower()

    match guess:
        case "quit":
            print(f"The word was {secret.upper()}.")
            break
        case "letters":
            print("Tried :", " ".join(sorted(used_letters)))
            continue

    if len(guess) != 5 or guess not in words:
        print("That is not a five-letter word I know.")
        continue

    guess_count += 1
    used_letters.update(guess)

    # [A] is in the right place, (A) is in the word elsewhere, A is not
in it.
    row = ""
    for position, letter in enumerate(guess):
        if letter == secret[position]:
            row += f"[{letter.upper()}]"
        elif letter in secret:
            row += f"({letter.upper()})"
        else:
            row += f" {letter.upper()} "
```

```
    print(row)

    if guess == secret:
        print(f"Got it in {guess_count}!")
        break
else:
    print(f"Out of guesses. The word was {secret.upper()}.")
```

The file is also in `solutions/04-control-flow/wordle.py`.

You have a game. Play a few rounds with a random secret : remove the environment variable, or start a new terminal.

If time permits

Add a third command, `hint`, which reveals one letter of the secret that the player has not found yet, in its place : for example `Letter 4 is N.` A player may use it once per game; a second `hint` says so instead. Think about what counts as found : a letter the player has already had marked `[A]` in that position is not worth revealing. There is no reference solution for this step.

Exercise 5 : Functions

Your Wordle works, but it is one long script from top to bottom. In this exercise you cut it into functions, and on the way you find and fix a bug that has been in the game since Exercise 2.

Continue in your own `wordle` project, or start from `exercises/starters/05-functions/`. The starter is the finished Exercise 4.

Some steps give you the code, some a skeleton to complete, and one only the function's signature and a few examples of what it should return. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : Find the bug

Play one round where you know the answer. The starter supports `WORDLE_SECRET`, an environment variable that overrides the random choice. In PowerShell, run :

```
$env:WORDLE_SECRET = "crane"
uv run python wordle.py
```

In Bash, use `WORDLE_SECRET=crane uv run python wordle.py`. If your own script does not have the override yet, temporarily set `secret = "crane"`.

Enter `error`, then `crane`. Here is the captured output, with typed input omitted :

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : (E)[R](R) O (R)
Guess 2/6 : [C][R][A][N][E]
Got it in 2!
```

CRANE has one R, and the green R in second place already uses it. The other two R's should be grey. Asking whether a letter occurs anywhere in the secret cannot tell one R from three. Keep the fixed secret until the end of the exercise.

Step 2 : A main function

Before the scoring can be fixed it needs to be something you can call on its own, and before anything can be called on its own the script must stop running a whole game the moment it is loaded.

Move the code that reads the word list into a function, and the rest of the script into a function called `main`. Call `main` at the bottom, behind a guard :

```
import os
import random
from pathlib import Path


def load_words(path: str = "words.txt") -> list[str]:
    """Read the word list, one word per line."""
    return Path(path).read_text(encoding="utf-8").split()


def main() -> None:
    words = load_words()
    secret = os.environ.get("WORDLE_SECRET", random.choice(words))
    ...  # the rest of the game, indented one level further


if __name__ == "__main__":
    main()
```

Run the game again with `uv run python wordle.py` and confirm it plays exactly as before.

The `if __name__ == "__main__":` line means : only start a game when this file is run, not when another file imports it. In Step 4 you import `wordle` into the interactive interpreter to try a function on its own, and without the guard the import would start a game and sit waiting for a guess. Module 6 explains how this works.

> ***Hint :*** *The slide Function Syntax shows the shape : everything indented under the `def` line is the body of the function. return Statement shows how `load_words` hands its list back.*

Step 3 : Checking a guess

The game loop rejects a guess that is the wrong length or not in the word list. Give that check a name. Add `WORD_LENGTH = 5` beside the imports, then fill in the body. See *Annotating a Function* for the signature :

```
def is_valid(guess: str, words: list[str]) -> bool:
    """A guess must be a known word of the right length."""
    return ...  # combine the length, letters-only and membership
checks
```

Then replace the check in the game loop with a call to it :

```
if not is_valid(guess, words):
    print("That is not a five-letter word I know.")
    continue
```

> ***Hint :*** *The slide Built-in String Methods, in module 02, lists a method that tells whether a string holds only letters. return Statement shows a function that returns the value of an expression.*

> ***Hint :*** *A word made of letters only is something a string method can tell you. And a condition is already `True` or `False`, so it can be returned as it is; there is no need for `if ... : return True` followed by `return False`.*

Solution on next page ↓

Solution

```
WORD_LENGTH = 5


def is_valid(guess: str, words: list[str]) -> bool:
    """A guess must be a known word of the right length."""
    return len(guess) == WORD_LENGTH and guess.isalpha() and guess in
words
```

`WORD_LENGTH` is a constant : a module-level name in capitals, which tells the reader it is not meant to change. Use it wherever the game says `5` .

Step 4 : Scoring a guess

This is the function the bug lives in. Here is its signature :

```
def score_guess(secret: str, guess: str) -> list[str]:
    """Mark each letter of guess as "green", "yellow" or "grey"."""
```

The rules :

- a letter in the right place is `"green"`
- a letter that is in the secret, but somewhere else, is `"yellow"`
- anything else is `"grey"`
- each letter of the secret can colour **one** letter of the guess, and a green match uses it up first

Here is what it should return for three guesses. The first is the bug from Step 1 : only the R in the right place is coloured. In the last, SPEED has two E's, so GEESE may colour two of its three and no more :

```
score_guess("crane", "error")   # ['yellow', 'green', 'grey', 'grey',
'grey']
score_guess("crane", "react")   # ['yellow', 'yellow', 'green',
'yellow', 'grey']
score_guess("speed", "geese")   # ['grey', 'yellow', 'green', 'yellow',
'grey']
```

Try your function in the interactive interpreter, the slide *Python in Interactive Mode*. Start it with `uv run python`, import the function, and call it :

```
>>> from wordle import score_guess
>>> score_guess("crane", "error")
['yellow', 'green', 'grey', 'grey', 'grey']
```

`from wordle import score_guess` makes the function in `wordle.py` available in the interpreter; module 06 explains it. The interpreter reads the file once, so after a change leave it with `exit()` and start it again. The game itself still uses its old scoring until you connect the new function in Step 5.

> ***Hint :*** *The slide List Operations builds a list by repetition,* `['go']*3`*, and List Functions and Methods has* `append` *and* `remove`*. Iterating with enumerate, in module 04, gives the position with each letter.*

> ***Hint :*** *The last rule is hard to get right in a single loop. Two loops make it easy. The first finds the greens, and keeps a list of the secret's letters that were not matched. The second goes over the remaining letters of the guess and marks one yellow only if its letter is still in that list, removing it when it does.*

Solution on next page ↓

Solution

```
def score_guess(secret: str, guess: str) -> list[str]:
    """Mark each letter of guess as "green", "yellow" or "grey".

    A letter is yellow only while the secret still has an unmatched
copy of it,
    so one E in the secret cannot make two E's in the guess yellow.
    """
    marks = ["grey"] * WORD_LENGTH
    unmatched: list[str] = []

    # First pass : exact matches. Whatever the secret has left over is
    # available to be found elsewhere.
    for position, letter in enumerate(guess):
        if letter == secret[position]:
            marks[position] = "green"
        else:
            unmatched.append(secret[position])

    # Second pass : right letter, wrong place, each copy used at most
once.
    for position, letter in enumerate(guess):
        if marks[position] != "green" and letter in unmatched:
            marks[position] = "yellow"
            unmatched.remove(letter)

    return marks
```

`["grey"] * WORD_LENGTH` builds a list of five `"grey"` strings, so every letter starts grey and only the two loops change that.

Step 5 : Drawing a row

The code that builds `[A]`, `(A)` and `A` now has the marks to work from instead of working them out itself. Move it into a function :

```
def render(guess: str, marks: list[str]) -> str:
    """[A] is in the right place, (A) is in the word elsewhere, A is
not in it."""
    row = ""
    for position, letter in enumerate(guess.upper()):
        ...
    return row
```

> ***Hint :*** *Your Exercise 4 code built this row already. The slide match Statement shows a* `case` *for each literal value and the wildcard* `case _` *for the rest.*

> ***Hint :*** *A* `match` *on* `marks[position]` *reads well here.*

The game loop then shrinks to a single line where the scoring used to be :

```
print(render(guess, score_guess(secret, guess)))
```

In the interpreter,
`render("react", ["yellow", "yellow", "green", "yellow", "grey"])` now returns `'(R)(E)[A](C) T '`.

Solution on next page ↓

Solution

```
def render(guess: str, marks: list[str]) -> str:
    """[A] is in the right place, (A) is in the word elsewhere, A is
not in it."""
    row = ""
    for position, letter in enumerate(guess.upper()):
        match marks[position]:
            case "green":
                row += f"[{letter}]"
            case "yellow":
                row += f"({letter})"
            case _:
                row += f" {letter} "
    return row
```

Step 6 : One game, as a function

What is left in `main` is the game itself. Move it into a function that plays one game and reports whether it was won :

```
def play(secret: str, words: list[str], max_guesses: int = 6) -> bool:
    """Play one game. Returns True when the word was found."""
    used_letters: set[str] = set()
    guess_count = 0
    ...  # move the loop here, returning True on a win and False
otherwise
```

`max_guesses=8` allows an easier game; the default is six. See *Default Arguments*. Keep loading the words and choosing the secret in `main`, then call `play(secret, words)`. Without `WORDLE_SECRET`, the secret is random.

> ***Hint :*** *The slide return Statement shows that `return` exits the function at once, so a `return` inside the loop ends the loop as well.*

> ***Hint :*** *Return `True` on a win, replacing the loop's `break` and `else`. In the `quit` case, keep only `break`; remove its old print so the secret is not shown twice. After the loop, reveal the secret and return `False`. Exercise 10 will use this result for statistics.*

Solution on next page ↓

Solution

```
def play(secret: str, words: list[str], max_guesses: int = 6) -> bool:
    """Play one game. Returns True when the word was found."""
    used_letters: set[str] = set()
    guess_count = 0

    while guess_count < max_guesses:
        guess = input(f"Guess {guess_count + 1}/{max_guesses} :
").strip().lower()

        match guess:
            case "quit":
                break
            case "letters":
                print("Tried :", " ".join(sorted(used_letters)))
                continue

        if not is_valid(guess, words):
            print("That is not a five-letter word I know.")
            continue

        guess_count += 1
        used_letters.update(guess)
        print(render(guess, score_guess(secret, guess)))

        if guess == secret:
            print(f"Got it in {guess_count}!")
            return True

    print(f"The word was {secret.upper()}.")
    return False
```

Replace `main` with :

```
def main() -> None:
    words = load_words()
    print("Guess the five-letter word. Type 'quit' to give up,
'letters' for the letters tried.")
    secret = os.environ.get("WORDLE_SECRET", random.choice(words))
    play(secret, words)
```

The complete file is in `solutions/05-functions/wordle.py`.

Play with `error`, then `crane`, against the fixed secret. The captured output now shows only one coloured R :

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : (E)[R] R  O  R
Guess 2/6 : [C][R][A][N][E]
Got it in 2!
```

Remove the override with `Remove-Item Env:WORDLE_SECRET` in PowerShell, or `unset WORDLE_SECRET` if you exported it in Bash, and play a random game.

If time permits

Add a hard mode, in which every guess must use what the earlier guesses revealed. A green letter must stay in its place, and a yellow letter must appear somewhere. Write

```
def follows_hints(guess: str, history: list[tuple[str, list[str]]]) ->
bool:
```

where `history` holds every earlier guess with its marks as `(guess, marks)` pairs, which `play` has to keep as a list. Add a parameter `hard: bool = False` to `play`, and reject a guess that breaks the rule when it is `True`. Try guesses that obey and break each rule. There is no reference solution for this step.

Exercise 6 : Modules

The scoring can now be called on its own, but loading words, drawing rows and running the game still share one file. Split those responsibilities into a package so the solver in the next exercise has a clear place to live.

Continue in your own `wordle` project, or use `exercises/starters/06-modules/`, the finished Exercise 5. Run commands from the project root, beside `pyproject.toml` and `words.txt`, not from inside the new package directory.

Step 1 : Start a package

Create a directory named `wordle` inside your project and an empty `wordle/__init__.py`. That file marks the directory as a package; see *Packages in Python*. Copy `load_words` into `wordle/words.py` with its import :

```
from pathlib import Path


def load_words(path: str = "words.txt") -> list[str]:
    """Read the word list, one word per line."""
    return Path(path).read_text(encoding="utf-8").split()
```

Keep the original `wordle.py` intact for this step so `uv run python wordle.py` still plays a game. In the next step you will finish the package and remove that old file. Leave `words.txt` beside `pyproject.toml`; moving the loader does not change the working directory.

Step 2 : Give each module one job

Move the existing functions into the following files. Keep their signatures and bodies; this is a change of organisation, not of scoring rules.

File	What belongs there
`wordle/scoring.py`	`WORD_LENGTH`, `is_valid`, `score_guess`
`wordle/display.py`	`render`
`wordle/game.py`	`play`
`wordle/__main__.py`	`main` and the `if __name__ == "__main__":` guard

Fill the import slots at the top of `game.py` :

```
from wordle.display import ...
from wordle.scoring import ...

# Move play here, keeping its body and annotations.
```

Use this skeleton in `__main__.py` :

```
import os
import random

from wordle.game import ...
from wordle.words import ...


def main() -> None:
    words = ...
    ...  # keep the welcome message and call play with the chosen
secret


if __name__ == "__main__":
    main()
```

Delete the old `wordle.py` once the package is complete. Check it with `uv run python -m wordle`.

> ***Hint :*** *The slide from…import Statement shows the import form, and Running Packages with python -m shows a package with a* `__main__.py` *like this one.*

> ***Hint :*** `game.py` *needs the renderer and both scoring functions. Neither of those modules imports the game.*

Solution on next page ↓

Solution

The complete moved functions are in `solutions/06-modules/wordle/`. The functions retain their Exercise 5 behaviour. These are the imports in `game.py` :

```
from wordle.display import render
from wordle.scoring import is_valid, score_guess
```

`__main__.py` contains :

```
import os
import random

from wordle.game import play
from wordle.words import load_words


def main() -> None:
    """Load the words and play one game."""
    words = load_words()
    print("Guess the five-letter word. Type 'quit' to give up,
'letters' for the letters tried.")
    play(os.environ.get("WORDLE_SECRET", random.choice(words)), words)


if __name__ == "__main__":
    main()
```

Step 3 : Run the package

Run from the project root :

```
uv run python -m wordle
```

The `-m` option asks Python to locate a module or package. For a package, Python runs `wordle/__main__.py`. The guard moved with the entry point; importing `wordle.scoring` does not start a game.

For a repeatable check in PowerShell, run :

```
$env:WORDLE_SECRET = "crane"
uv run python -m wordle
```

In Bash, use `WORDLE_SECRET=crane uv run python -m wordle`. Enter `slate`, then `crane`. Captured output, with typed input omitted :

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 :  S  L [A] T [E]
Guess 2/6 : [C][R][A][N][E]
Got it in 2!
```

Step 4 : Import from the new modules

In Exercise 5 you imported `score_guess` from `wordle.py` into the interactive interpreter. That file has gone, so the functions are now imported from the modules that hold them. Fill these two import lines yourself :

```
from wordle.scoring import ...
from wordle.display import ...
```

Start the interpreter from the project root with `uv run python`, type your two imports, and score and draw the guess from Step 3 :

```
>>> score_guess("crane", "slate")
['grey', 'grey', 'green', 'grey', 'green']
>>> render("slate", score_guess("crane", "slate"))
' S  L [A] T [E]'
```

> ***Hint :*** *The slide Importing Names from a Package shows* `from cameras.canon import canon_info` *: the package, a dot, the module, then the name.*

> ***Hint :*** *Import each function from the module that defines it, not from* `wordle` *itself.*

> *An empty `__init__.py` does not automatically import the other modules.*

Solution on next page ↓

Solution

```
from wordle.scoring import score_guess
from wordle.display import render
```

Your project now has this shape, apart from environment and lock files :

```
wordle/
    pyproject.toml
    words.txt
    wordle/
        __init__.py
        __main__.py
        words.py
        scoring.py
        display.py
        game.py
```

Remove the fixed-secret override with `Remove-Item Env:WORDLE_SECRET` in PowerShell, or `unset WORDLE_SECRET` if exported in Bash. Play another game with `uv run python -m wordle`. Its behaviour should be the same as Exercise 5, with the duplicate-letter fix intact.

If time permits

Try colour as an optional display experiment. Install `rich` with `uv add rich`, then use its printing support to colour a row when displaying it. Keep the text returned by `render` unchanged, so that the rest of the game does not notice. See *Installing Packages* for dependencies.

The following exercises and reference solutions continue with the text markers; colour is not required to proceed.

Exercise 7 : Comprehensions

Your game can judge a guess, but it cannot yet use the feedback to choose its next move. In this exercise you turn the same scoring function into a solver: it will rule out impossible answers and recommend a guess from the words that remain.

Continue in your own `wordle` project, or use `exercises/starters/07-comprehensions/`, the finished Exercise 6. Keep running commands beside `pyproject.toml`.

Step 1 : Build a row from expressions

In `wordle/display.py`, replace the loop with a joined sequence. Complete the call for each letter and mark :

```
def render(guess: str, marks: list[str]) -> str:
    symbols = {
        "green": lambda letter: f"[{letter}]",
        "yellow": lambda letter: f"({letter})",
        "grey": lambda letter: f" {letter} ",
    }
    return "".join(... for letter, mark in zip(guess.upper(), marks,
strict=True))
```

See *Zip Function*. `strict=True` rejects sequences of different lengths. The row must come out exactly as before. Check it in the interpreter, as in Exercise 6 :

```
>>> from wordle.display import render
>>> render("react", ["yellow", "yellow", "green", "yellow", "grey"])
'(R)(E)[A](C) T '
```

> ***Hint :*** *The slide Generator Expressions shows an expression written straight inside a call, and Functions in Dictionaries, in module 05, shows a dictionary of functions like* `symbols` *and how to call one.*

> ***Hint :*** *Call* `symbols[mark]` *with* `letter` *;* `join` *concatenates the results.*

Solution on next page ↓

Solution

```
def render(guess: str, marks: list[str]) -> str:
    """Draw green letters in brackets, yellow in parentheses and grey
padded."""
    symbols = {
        "green": lambda letter: f"[{letter}]",
        "yellow": lambda letter: f"({letter})",
        "grey": lambda letter: f" {letter} ",
    }
    return "".join(symbols[mark](letter) for letter, mark in
zip(guess.upper(), marks, strict=True))
```

The expression inside `join` is a generator expression: it produces values as they are consumed, without building a separate list. You will use the same form inside `all` and `sum`; writing generators is covered further in Advanced Python.

Step 2 : Turn the scorer around

Imagine that a candidate word really is the secret. Would it give exactly the feedback you saw? If it would, the word is still possible. This reuses the duplicate-letter rules you already wrote, rather than inventing another set of rules for the solver.

Create `wordle/solver.py` with :

```
from wordle.scoring import score_guess


def consistent(word: str, guess: str, marks: list[str]) -> bool:
    """Return whether word would produce the observed marks for
guess."""
    return score_guess(word, guess) == marks
```

Try it in the interpreter. Score `slate` against the secret `crane` ; `crane` would give those marks, but a guess of `raise` would not :

```
>>> from wordle.scoring import score_guess
>>> from wordle.solver import consistent
>>> marks = score_guess("crane", "slate")
>>> consistent("crane", "slate", marks)
True
>>> consistent("crane", "raise", marks)
False
```

The game remains playable while you build its solver.

Step 3 : Keep every answer that fits

Add this function to `solver.py` :

```
def candidates(words: list[str], history: list[tuple[str, list[str]]])
-> list[str]:
    """Return words consistent with every guess and its marks."""
```

`history` contains pairs of a guess and its marks. Keep a word only if it agrees with every pair, preserving the order of `words` . An empty history rules out nothing.

After `slate` was scored against `crane` , only `crane` of these three words still fits; with no history, all three do :

```
marks = score_guess("crane", "slate")
candidates(["crane", "slate", "trace"], [("slate", marks)])   #
['crane']
candidates(["crane", "slate", "trace"], [])                   #
['crane', 'slate', 'trace']
```

Try them in the interpreter, importing `candidates` from `wordle.solver` and `score_guess` from `wordle.scoring`.

> ***Hint :*** *The slide Filtered List Comprehension shows a comprehension that keeps only some items, and Generator Expressions shows* `all` *.*

> ***Hint :*** *Use an outer list comprehension over* `words` *and an* `all(...)` *condition over the pairs in* `history` *.* `all` *is true for an empty sequence, which gives the required empty-history behaviour automatically.*

Solution on next page ↓

Solution

```
def candidates(words: list[str], history: list[tuple[str, list[str]]])
-> list[str]:
    """Return words consistent with every guess and its marks."""
    return [
        word
        for word in words
        if all(consistent(word, guess, marks) for guess, marks in
history)
    ]
```

Step 4 : Choose a useful next guess

Prefer an answer whose distinct letters occur in many remaining words. This heuristic favours useful information; it does not guarantee the fewest guesses.

Complete these functions in `solver.py`. Count words containing each letter; repeated occurrences in one word contribute only once.

```
def letter_counts(words: list[str]) -> dict[str, int]:
    letters = {... for word in words for letter in word}
    return {letter: ... for letter in letters}


def best_guess(words: list[str]) -> str:
    counts = letter_counts(words)
    return max(words, key=lambda word: ...)
```

`best_guess` needs a non-empty list; during normal play the secret remains a candidate. On a tie, `max` chooses the first word with that score.

> ***Hint :*** *The slides Set Comprehensions and Dictionary Comprehensions show the two comprehensions, Generator Expressions shows* `sum` *over a generator, and Built-ins that Take a Function shows* `max` *with a* `key` *.*

> ***Hint :*** *For each letter, sum* `letter in word` *over the words. Booleans contribute 1 or 0. For a proposed guess, sum the counts for* `set(word)` *so repeating a letter does not inflate its score.*

Solution on next page ↓

Solution

```
def letter_counts(words: list[str]) -> dict[str, int]:
    """Count how many candidate words contain each letter."""
    letters = {letter for word in words for letter in word}
    return {letter: sum(letter in word for word in words) for letter in
letters}
```

```
def best_guess(words: list[str]) -> str:
    """Choose the word with the most frequently occurring distinct
letters."""
    counts = letter_counts(words)
    return max(words, key=lambda word: sum(counts[letter] for letter in
set(word)))
```

Try both in the interpreter. CACAO has two A's but counts once for `a`, and its repeated letters earn it no extra score :

```
counts = letter_counts(["cacao", "crane"])
counts["a"]                          # 2 : each word counts once
counts["o"]                          # 1
best_guess(["cacao", "crane"])       # 'crane'
```

Step 5 : Ask the game for help

Add `from wordle.solver import best_guess, candidates` to `wordle/game.py`. Initialise `history` as an empty list beside `used_letters` inside `play`. After scoring each valid guess, save `(guess, marks)` before rendering the row. Commands and rejected guesses must not enter the history.

Extend the existing `match guess` using this skeleton, keeping the existing `quit` and `letters` cases :

```
match guess:
    case "hint":
        possible = ...
        print(...)  # number of candidates and the suggested word in
upper case
        continue
```

Use the message format shown in the output below. Requesting a hint must not use a turn. After implementing it, set `WORDLE_SECRET` to `crane` and run the game :

```
$env:WORDLE_SECRET = "crane"
uv run python -m wordle
```

In Bash, use `WORDLE_SECRET=crane uv run python -m wordle`. Enter `slate`, `hint`, then `crane`.

> ***Hint :*** *This is Exercise 5's* `play` *with one more* `case`*; the slide match Statement shows the form, and List Functions and Methods has* `append`*.*

> ***Hint :*** *Calculate the marks once, append the pair to* `history`*, and pass those same marks to* `render`*. The hint case calls* `candidates(words, history)` *first, then* `best_guess(possible)`*.*

Expected output :

Captured output with typed input omitted. Notice that the hint leaves the prompt at guess 2.

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 :  S  L [A] T [E]
Guess 2/6 : 15 words left, try GRACE
Guess 2/6 : [C][R][A][N][E]
Got it in 2!
```

Solution on next page ↓

Solution

Add the history beside the existing initialisation :

```
history = []
```

Add this case to the existing command handling :

```
match guess:
    case "hint":
        possible = candidates(words, history)
        print(f"{len(possible)} words left, try
{best_guess(possible).upper()}")
        continue
```

After `guess_count += 1` and `used_letters.update(guess)`, replace the old scoring-and-printing line with :

```
marks = score_guess(secret, guess)
history.append((guess, marks))
print(render(guess, marks))
```

The complete integration is in `solutions/07-comprehensions/wordle/game.py`.

Remove the fixed-secret override with `Remove-Item Env:WORDLE_SECRET` in PowerShell, or `unset WORDLE_SECRET` if exported in Bash. Play a random game and use `hint` after each guess. The suggestion comes from your scoring rules and the feedback recorded so far; the solver never reads the secret directly.

If time permits

Write `solve(secret, words)` to play a whole game automatically. Start with an empty history, choose with `best_guess`, score the guess, append its feedback, and filter again until the guess is correct. Return the number of guesses used. Try the first 100 words as secrets and calculate the average. Keep this experiment separate from the interactive game.

Exercise 8 : Exception Handling

The game gives the same rejection for every bad guess, and a missing word list or an interrupted input ends in a traceback. Give each failure a useful response.

Continue in your own `wordle` project, or use `exercises/starters/08-exception-handling/`, the finished Exercise 7.

Step 1 : Name an invalid guess

Add this class below `WORD_LENGTH` in `wordle/scoring.py` :

```
class InvalidGuess(ValueError):  # noqa: N818  # Contracted name from plan section 4.
    """Raised when a guess is not an allowed five-letter word."""
```

This creates a specific `ValueError` the game can catch. See *User Defined Exceptions*; Module 11 covers classes in detail. The `noqa` comment permits the course's exception name instead of Ruff's preferred `Error` suffix.

Next, make the validator raise this exception with a reason.

Step 2 : Explain why a guess is invalid

Add `check_guess` to `scoring.py`. Check length, letters, then membership. Valid guesses return normally; invalid ones raise `InvalidGuess`.

```
def check_guess(guess: str, words: list[str]) -> None:
    if ...:
        raise InvalidGuess("A guess must have five letters.")
    if ...:
        raise InvalidGuess("A guess must contain letters only.")
    if ...:
        raise InvalidGuess("That word is not in the word list.")
```

Keep `is_valid` so the current game still works, but make it a wrapper around the new validator :

```
def is_valid(guess: str, words: list[str]) -> bool:
    try:
        ...  # call the validator
    except InvalidGuess:
        ...  # report failure as a boolean
    return True
```

Try the validator in the interpreter. A valid guess returns `None`; each invalid one stops with a traceback whose last line gives its reason, and `is_valid` turns the same failure into `False` :

```
>>> from wordle.scoring import check_guess, is_valid
>>> print(check_guess("crane", ["crane", "slate"]))
None
>>> check_guess("cran", ["crane", "slate"])
Traceback (most recent call last):
  ...
wordle.scoring.InvalidGuess: A guess must have five letters.
>>> check_guess("cr4ne", ["crane", "slate"])
Traceback (most recent call last):
  ...
wordle.scoring.InvalidGuess: A guess must contain letters only.
>>> is_valid("zzzzz", ["crane", "slate"])
False
```

Hint : *The slide Raising Exceptions shows `raise` with a message, and Typed Exception Handling catches one exception type by name.*

Hint : *Compare `len(guess)` with `WORD_LENGTH`, use `guess.isalpha()`, and then test membership in `words`. Inside the wrapper, catch only `InvalidGuess` and return `False`; unrelated programming errors should remain visible.*

Solution on next page ↓

Solution

```
def check_guess(guess: str, words: list[str]) -> None:
    """Raise InvalidGuess with a reason when guess cannot be played."""
    if len(guess) != WORD_LENGTH:
        raise InvalidGuess("A guess must have five letters.")
    if not guess.isalpha():
        raise InvalidGuess("A guess must contain letters only.")
    if guess not in words:
        raise InvalidGuess("That word is not in the word list.")
```

```
def is_valid(guess: str, words: list[str]) -> bool:
    """Return whether guess is valid, for compatibility with Exercise
6."""
    try:
        check_guess(guess, words)
    except InvalidGuess:
        return False
    return True
```

Step 3 : Recover without spending a turn

In `wordle/game.py`, replace the scoring import with :

```
from wordle.scoring import InvalidGuess, check_guess, score_guess
```

Replace `if not is_valid(...)` after command handling, before updating the guess count or history :

```
try:
    ...  # validate the guess
except InvalidGuess as error:
    ...  # display the reason
    continue
```

> ***Hint :*** *The slide Example Argument of an Exception catches an exception `as error` and prints it; Exception Arguments says `print(error)` shows the message.*

> ***Hint :*** *Call* `check_guess(guess, words)` *; catch and print* `InvalidGuess` *.*

Solution on next page ↓

Solution

```
try:
    check_guess(guess, words)
except InvalidGuess as error:
    print(error)
    continue
```

Step 4 : Stop cleanly when play cannot continue

In `wordle/__main__.py`, wrap loading the words and playing the game in handlers for a missing file and interrupted input. Keep `os`, `random`, `play` and `load_words` imported, and keep the existing guard below `main`.

```
def main() -> None:
    try:
        ...  # load words, print the welcome message and play
    except FileNotFoundError:
        ...  # explain how to restore the word list
    except (KeyboardInterrupt, EOFError):
        ...  # print a newline followed by a friendly stopping message
```

For a missing file, print `The word list is missing. Restore words.txt and try again.` For interrupted input, print `\nGame stopped.` Try Ctrl+C while the game waits for a guess. End-of-input is Ctrl+D on Unix terminals, or Ctrl+Z followed by Enter on Windows. To try the missing-file case, temporarily rename `words.txt`, run the game, and restore the filename immediately afterwards.

> ***Hint :*** *The slide Typed Exception Handling shows several `except` clauses after one `try`, and except Clause Multiple Exceptions catches a tuple of types in one clause.*

> ***Hint :*** *The game can recover from one bad guess, so that handler belongs inside `play`. Missing data or interrupted input ends the session, so those handlers belong in `main`. A tuple lets one handler catch either interruption type.*

Solution on next page ↓

Solution

```
def main() -> None:
    """Load the words and play one game."""
    try:
        words = load_words()
        print(
            "Guess the five-letter word. Type 'quit' to give up, "
            "'letters' for the letters tried."
        )
        play(os.environ.get("WORDLE_SECRET", random.choice(words)),
words)
    except FileNotFoundError:
        print("The word list is missing. Restore words.txt and try
again.")
    except (KeyboardInterrupt, EOFError):
        print("\nGame stopped.")
```

Finally, set `WORDLE_SECRET` to `crane` and play again :

```
$env:WORDLE_SECRET = "crane"
uv run python -m wordle
```

In Bash, use `WORDLE_SECRET=crane uv run python -m wordle`. Enter `cran`, `cr4ne`, `zzzzz`, then `crane`. Captured output, with typed input omitted :

Expected output :

```
Guess the five-letter word. Type 'quit' to give up, 'letters' for the
letters tried.
Guess 1/6 : A guess must have five letters.
Guess 1/6 : A guess must contain letters only.
Guess 1/6 : That word is not in the word list.
Guess 1/6 : [C][R][A][N][E]
Got it in 1!
```

All three invalid guesses leave the first turn available. Remove the override with `Remove-Item Env:WORDLE_SECRET` in PowerShell, or `unset WORDLE_SECRET` if exported in Bash, and play a random game. The solver's `hint` command should still work.

If time permits

Try a too-long guess and an input that fails more than one rule, such as `cr4n`, in the game and in the interpreter. Decide which reason should take priority and check that the implementation follows that decision. Keep the game playable after each change.

Exercise 9 : Python IO

Your game plays well, but it lives on borrowed data and it has no memory. The word list is a file you copied in Exercise 1, it is only found when you start the game from the right folder, and the moment a game ends the result is gone. In this exercise you clean a real word list, make the game find its data wherever it is started from, and keep a history of every game on disk.

Continue in your own `wordle` project, or start from `exercises/starters/09-python-io/`. The starter is the finished Exercise 8 plus this exercise's handouts : `raw-words.txt`, `allowed.txt` and `SOURCES.md`. If you continue in your own project, copy those three files in from `exercises/handouts/09-python-io/`.

Some steps give you the code, some a skeleton to complete, and some only a description of what to write. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : Clean a messy word list

`raw-words.txt` is what a word list looks like when it comes from the outside world : almost a thousand lines with some words in capitals, some duplicates, a few lines of the wrong length, a number, a word with an accent, and an empty line. Open it in the editor and scroll to the bottom to see for yourself.

Your game wants one word per line : lower case, five letters, ASCII letters only, no duplicates, in alphabetical order. Write a function in `wordle/words.py` that turns the lines of a file into such a list :

```
def clean_words(lines: list[str]) -> list[str]:
    """Return unique lowercase five-letter ASCII words in sorted
order."""
    kept = set()
    for line in lines:
        word = line.strip().lower()
        if ...:
            kept.add(word)
    return ...
```

Replace each `...` : the condition that keeps a word, and the sorted list that is returned. `str.isalpha()` is also `True` for letters with an accent, so a five-letter word such as `étage` would pass it; `str.isascii()` is `True` only when every character is plain ASCII, which rules it out.

Try the function in the interpreter, as in Exercise 8. Capitals, duplicates and empty lines disappear, and so do words of the wrong length, numbers and accents :

```
clean_words(["Slate", "crane", "CRANE", ""])              # ['crane',
'slate']
clean_words(["cranes", "12345", "étage", " adieu "])      # ['adieu']
```

Hint : *The slide Built-in String Methods, in module 02, has `strip`, `lower` and `isalpha`, and Set Operations, in module 03, shows that a set keeps each value once and that `sorted` returns a list.*

Hint : *Strip the whitespace from each line before you look at it, then check its length, `isascii()` and `isalpha()`.*

Hint : *A set throws duplicates away and `sorted(...)` turns it back into a list. A set comprehension can hold the whole filter :* `{line.strip().lower() for line in lines if ...}`.

Solution on next page ↓

Solution

```
def clean_words(lines: list[str]) -> list[str]:
    """Return unique lowercase five-letter ASCII words in sorted
order."""
    return sorted(
        {
            line.strip().lower()
            for line in lines
            if len(line.strip()) == 5
            and line.strip().isascii()
            and line.strip().isalpha()
        }
    )
```

The condition strips each line three times, which is easy to read and fast enough for a thousand lines.

Step 2 : Write the clean list

Cleaning is done once, not every time the game starts, so use a throwaway session instead of a script. Start Python in the project folder and type this, line by line :

```
uv run python
```

```
from pathlib import Path
from wordle.words import clean_words

lines = Path("raw-words.txt").read_text(encoding="utf-8").splitlines()
words = clean_words(lines)
len(lines), len(words)

Path("wordle/data").mkdir(exist_ok=True)
Path("wordle/data/words.txt").write_text("\n".join(words) + "\n",
encoding="utf-8")
```

`splitlines()` cuts the text into a list with one line per item. The session answers with the number of lines read and the number of words kept. The last line returns the number of characters written. Leave Python with `exit()`.

```
(981, 935)
5610
```

The folder `wordle/data/` is inside the package, next to the code that reads it. Data that belongs to the program travels with the program. Now `wordle/data/words.txt` is the answer list, and the old `words.txt` in the project folder is no longer needed : delete it, so that nothing can keep using it by accident. It held the same 935 words.

Also copy the handed-out `allowed.txt` and `SOURCES.md` into `wordle/data/`. You use `allowed.txt` in Step 4; `SOURCES.md` records where the word lists come from.

Step 3 : Find the file from the package

Your `load_words` still opens `words.txt` as a path relative to wherever the program was started. Try what that means. Start Python in the project folder again and import the function first, because after the change of folder Python can no longer find the `wordle` package. Then change the working directory :

```
import os
from wordle.words import load_words

os.chdir("wordle")
load_words()
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'words.txt'
```

A relative path is resolved against the working directory, and the working directory belongs to whoever starts the program, not to the program. The same holds for a game started from a shortcut, from a scheduled task or from an editor in another folder. Leave this Python session with `exit()` before editing the loader.

The fix is to ask the module where it lives. Every module knows its own file name in `__file__`, and `pathlib` turns that into a folder you can build a path from. Replace `load_words` in `wordle/words.py`:

```
def load_words() -> list[str]:
    """Read the package answer list, independent of the working
directory."""
    data_path = Path(__file__).parent / "data" / "words.txt"
    return data_path.read_text(encoding="utf-8").split()
```

`Path(__file__).parent` is `wordle/`; `/` joins path parts (see *Paths with pathlib*). The loader now finds its own data, so the `path` parameter is gone. Start a fresh `uv run python` session from the project folder, repeat the import and `os.chdir("wordle")`, then evaluate `len(load_words())`:

```
935
```

Leave Python with `exit()`, then run `uv run python -m wordle` in the terminal. The game now finds its package data without a `words.txt` in the project folder.

One more thing still points at the old file. The `except FileNotFoundError` in `main` of `wordle/__main__.py` tells the player to restore `words.txt`, which you just deleted. The word lists now live in `wordle/data/`, and Step 4 adds a second one, so change the message to name both:

```
    except FileNotFoundError:
        print(
            "A word list is missing. Restore wordle/data/words.txt "
            "and wordle/data/allowed.txt and try again."
        )
```

Step 4 : Two lists, answers and allowed guesses

You have been using one list for two jobs : the words the game can pick as a secret, and the words it accepts as a guess. A player should not be told that a perfectly good word is not a word, but the secret should be a word they have a fair chance of knowing. The real game uses two lists for this, and now your project does too :

- `wordle/data/words.txt` : the 935 common words the game picks its secret from, and the words the `hint` command searches
- `wordle/data/allowed.txt` : 8 639 words the game accepts as a guess, including all of the answers

Add a second loader to `wordle/words.py`, next to `load_words` :

```
def load_allowed() -> list[str]:
    """Read every allowed guess from the package data folder."""
    ...
```

Then change `play` in `wordle/game.py`. Load the allowed list once, at the top of the function, and check guesses against it, instead of against `words`. Everything else keeps using `words`.

Play with `WORDLE_SECRET=crane` set. This is the environment variable that `main` has read since Exercise 5, and it fixes the secret word. In a bash shell, put it in front of the command : `WORDLE_SECRET=crane uv run python -m wordle`. In

PowerShell, set it once with `$env:WORDLE_SECRET = "crane"` and then run the command as usual. `crone` is a word the game used to reject, and it is accepted now :

```
[C][R] O [N][E]
```

> ***Hint :*** *The slide Paths with pathlib shows `/` joining the parts of a path. The body of `load_allowed` is `load_words` with another file name; you may copy it.*

> ***Hint :*** *The change in `play` is two lines : `allowed_words = load_allowed()` before the loop, and `check_guess(guess, allowed_words)` in the `try` block. `candidates(words, history)` for the `hint` command stays as it is.*

Solution on next page ↓

Solution

```
def load_allowed() -> list[str]:
    """Read every allowed guess from the package data folder."""
    data_path = Path(__file__).parent / "data" / "allowed.txt"
    return data_path.read_text(encoding="utf-8").split()
```

In `wordle/game.py` add `from wordle.words import load_allowed` and change the start of `play` and the validation :

```
def play(secret: str, words: list[str], max_guesses: int = 6) -> bool:
    """Play one game and return whether the player found the secret."""
    used_letters = set()
    history = []
    guess_count = 0
    allowed_words = load_allowed()
    while guess_count < max_guesses:
        ...
        try:
            check_guess(guess, allowed_words)
        except InvalidGuess as error:
            print(error)
            continue
```

Step 5 : Keep a history file

A finished game should leave a line behind. One line per game, such as `crane,2,1`, holds the secret, the number of guesses used and `1` for a win or `0` for a loss. That is a CSV file, and the `csv` module knows how to write one. Read *The csv Module* first.

Put the history file in the project folder, next to the `wordle/` package, and for the same reason as in Step 3 do not let it depend on the working directory. This function is given, add it to `wordle/game.py` together with `from pathlib import Path` and `import csv` at the top :

```
def history_path() -> Path:
    """Return the history file beside the project package."""
    return Path(__file__).resolve().parents[1] / "history.txt"
```

`Path(__file__).resolve()` is the full path of `game.py` ; `parents[0]` is the `wordle/` folder and `parents[1]` is the project folder. Now write the function that adds a line :

```
def record_history(secret: str, guesses: int, won: bool) -> None:
    """Append one finished game to the CSV history file."""
    with history_path().open(..., encoding="utf-8", newline="") as
history_file:
        ...
```

Open the file for **appending** with `"a"` , so earlier games stay. See *File Open Modes* and *Using with*. Inside the `with` , give `csv.writer` the row `[secret, guesses, int(won)]` . Then call `record_history` in `play` : once with `True` just before the `return True` of a win, and once with `False` after the loop, before the final `return False` . A game you give up with `quit` counts as lost.

Play two games with `WORDLE_SECRET=crane` : guess `slate` and then `crane` in the first, and `slate` followed by `quit` in the second. Open `history.txt` :

```
crane,2,1
crane,1,0
```

> ***Hint :*** *The slide The csv Module writes rows with `csv.writer` inside a `with` block, and File Open Modes lists `a` for appending.*

> ***Hint :*** *`csv.writer(history_file).writerow([...])` writes one row and adds the line ending itself. `newline=""` stops Python from adding its own line ending on top of the one the `csv` module writes.*

> ***Hint :*** *`won` is a `bool` . `int(True)` is `1` and `int(False)` is `0` , which makes the file easy for other tools to read.*

Solution on next page ↓

Solution

```
def record_history(secret: str, guesses: int, won: bool) -> None:
    """Append one finished game to the CSV history file."""
    with history_path().open("a", encoding="utf-8", newline="") as
history_file:
        csv.writer(history_file).writerow([secret, guesses, int(won)])
```

The calls in `play` :

```
        if guess == secret:
            print(f"Got it in {guess_count}!")
            record_history(secret, guess_count, True)
            return True
    print(f"The word was {secret.upper()}.")
    record_history(secret, guess_count, False)
    return False
```

Step 6 : A stats command

Reading the file back is the other half. Add a new command to `play` , next to `quit` , `letters` and `hint` , so that typing `stats` prints how many games were played and how many were won :

```
            case "stats":
                print_stats()
                continue
```

The function is given, because Exercise 10 replaces it with a database query. Add it to `wordle/game.py` :

```
def print_stats() -> None:
    """Read the CSV history and print game and win totals."""
    path = history_path()
    if not path.exists():
        print("Games played: 0; won: 0.")
        return
    with path.open(encoding="utf-8", newline="") as history_file:
        rows = list(csv.reader(history_file))
    wins = sum(row[2] == "1" for row in rows)
    print(f"Games played: {len(rows)}; won: {wins}.")
```

Read it line by line :

- The `return` on its own line leaves the function early when there is no history file yet, so the rest does not need an `else` .
- Every field read from a CSV file is a string, so the wins are the rows whose third column equals `"1"` , not the number `1` .
- `sum(...)` over conditions counts how many are true, because `True` is `1` .

Also change the welcome line in `wordle/__main__.py` so it lists the commands the game now knows :

```
        print(
            "Guess the five-letter word. Type 'quit', 'letters', "
            "'hint' or 'stats'."
        )
```

The game is playable again, with a memory. After the two games of Step 5, type `stats` at the first prompt of a new game :

```
Games played: 2; won: 1.
```

The complete project is in `solutions/09-python-io/` .

If time permits

Add a `last` command that prints the final five games from `history.txt`, newest first, one per line in the form `CRANE : won in 2` or `CRANE : lost`. Slicing and `reversed()` do the work. There is no reference solution for this step.

Exercise 10 : Database Access

The history file of Exercise 9 can tell you how many games you played, but not much more. What is your win rate? In how many guesses do you usually win? How many games in a row have you won? Every question means another loop over the file. A database answers them in one query. In this exercise you move the statistics into SQLite and let the game report them. The SQL statements are given, because the course is about Python : your work is the Python around them, which passes the values in, fetches the rows and turns them into numbers, a bar chart and a streak.

Continue in your own `wordle` project, or start from `exercises/starters/10-database-access/`. The starter is the finished Exercise 9; this exercise hands out nothing.

Some steps give you the code, some a skeleton to complete, and some only a description and a few examples of what it should return. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : Open the database

Create `wordle/stats.py`, the module that owns everything to do with the database :

```
"""Store and summarize completed games in SQLite."""

import sqlite3
from datetime import datetime


def connect(path: str = "wordle.db") -> sqlite3.Connection:
    """Open a database and create the games table when needed."""
    connection = sqlite3.connect(path)
    connection.execute(
        """CREATE TABLE IF NOT EXISTS games (
            id INTEGER PRIMARY KEY,
            played_at TEXT,
            secret TEXT,
            guesses INTEGER,
            won INTEGER
        )"""
    )
    return connection
```

`sqlite3.connect` opens the file, and creates it when it does not exist. `CREATE TABLE IF NOT EXISTS` makes the function safe to call at the start of every game : the first call makes the table, later calls find it. SQLite has no `BOOLEAN`, so `won` is an `INTEGER` holding `0` or `1`, the same convention as the history file. Read *DB API Concepts* if you want the picture of connection, cursor and result. Creating the table does not need a `commit()` ; only statements that change rows do, and the next step says who commits those.

Check it in a Python session :

```
uv run python
```

```
from wordle.stats import connect

connection = connect()
connection.execute("SELECT name FROM sqlite_master").fetchall()
connection.close()
```

```
[('games',)]
```

A connection holds the file open until you close it. Closing it explicitly is the habit to keep, and the last step of this exercise makes it automatic.

There is now a file `wordle.db` in the project folder. Like the `words.txt` of Exercise 9 used to be, it is created in the working directory, so start the game from the project folder. Finding a fixed location for it is left to the last step, If time permits.

Step 2 : Record a finished game

Add a function that stores one game. The time is given, in the ISO text format SQLite sorts correctly :

```
def record_game(connection: sqlite3.Connection, secret: str, guesses:
int, won: bool) -> None:
    """Insert one game using a parameterized statement; the caller
commits."""
    played_at = datetime.now().isoformat(timespec="seconds")
    ...
```

Insert a row with the time, the secret, the number of guesses and the result. Do not build the SQL text with an f-string, however convenient. Write `?` for each value in the statement and pass the values as a tuple to `execute` ; the driver then handles quoting and nobody can sneak SQL in through a value. See *Inserting Data* and *Parameterized Queries*. The function does **not** commit. A transaction has exactly one owner : the code that calls `record_game` decides when the work becomes permanent, so that several inserts can succeed or fail together. Step 6 shows the caller. See *Committing Data* and *Transactions*.

Try it in the Python session :

```
from wordle.stats import connect, record_game

connection = connect()
record_game(connection, "crane", 2, True)
connection.execute("SELECT * FROM games").fetchall()
connection.close()
```

```
[(1, '2026-10-17T10:08:15', 'crane', 2, 1)]
```

The `id` is filled in by the database, and your time is of course a different one. Nothing was committed, so closing the connection threw the row away; this experiment leaves no game behind in `wordle.db`.

> ***Hint :*** *The slide Parameterized Queries writes `?` in the statement and passes the values as a tuple after it.*

> ***Hint :*** *The statement is `INSERT INTO games (played_at, secret, guesses, won) VALUES (?, ?, ?, ?)`. The database has no `bool`, so pass `int(won)`.*

Solution on next page ↓

Solution

```
def record_game(connection: sqlite3.Connection, secret: str, guesses:
int, won: bool) -> None:
    """Insert one game using a parameterized statement; the caller
commits."""
    played_at = datetime.now().isoformat(timespec="seconds")
    connection.execute(
        "INSERT INTO games (played_at, secret, guesses, won) VALUES (?,
?, ?, ?)",
        (played_at, secret, guesses, int(won)),
    )
```

Step 3 : Totals

Now ask the database a question. Write `summary`, which returns a dictionary with how many games were played, how many were won and the win percentage :

```
def summary(connection: sqlite3.Connection) -> dict:
    """Return totals, guess distribution and the current winning
streak."""
    played, won = connection.execute(
        "SELECT COUNT(*), COALESCE(SUM(won), 0) FROM games"
    ).fetchone()
    ...
```

The docstring already names all that the function will return after Step 5. For now the dictionary has three keys : `"played"`, `"won"` and `"win_percentage"`.

The query is in the skeleton. `COUNT(*)` counts the rows of the table, and `SUM(won)` adds up the `won` column, so the `1`s are the wins. One query gives both numbers as a single result row. On an empty table `SUM` has nothing to add and returns `NULL`, which Python sees as `None` ; `COALESCE(x, 0)` replaces a `NULL` by `0`. See *Filtering and Grouping Data*. What is left for you is the Python : `fetchone()` returns that row as a tuple, which the skeleton unpacks into two names, and then you build the dictionary and work out the percentage.

With no games the percentage would divide by zero. Divide only when there are games :

```
percentage = won / played * 100 if played else 0.0
```

This is a conditional expression : it is the value before `if` when the condition is true, and the value after `else` when it is not.

Try it in a Python session on a throwaway database. `connect(":memory:")` builds one in memory, which disappears when the connection does; the slide *Last inserted row id* uses one too. Record four games, three of them won, and ask for the totals; then ask an empty database :

```
from wordle.stats import connect, record_game, summary

connection = connect(":memory:")
record_game(connection, "crane", 2, True)
record_game(connection, "crane", 6, False)
record_game(connection, "crane", 4, True)
record_game(connection, "crane", 4, True)
summary(connection)
summary(connect(":memory:"))
```

```
{'played': 4, 'won': 3, 'win_percentage': 75.0}
{'played': 0, 'won': 0, 'win_percentage': 0.0}
```

Hint : *The slide Retrieving Data shows `fetchone()`, and Dictionaries, in module 03, builds a dictionary from keys and values.*

Hint : `played, won = connection.execute("...").fetchone()` *unpacks the one result row into two names.*

Solution on next page ↓

Solution

```
def summary(connection: sqlite3.Connection) -> dict:
    """Return totals, guess distribution and the current winning
streak."""
    played, won = connection.execute(
        "SELECT COUNT(*), COALESCE(SUM(won), 0) FROM games"
    ).fetchone()
    percentage = won / played * 100 if played else 0.0
    return {
        "played": played,
        "won": won,
        "win_percentage": percentage,
    }
```

Step 4 : The guess distribution

How quickly do you win? Add a key `"distribution"` to the dictionary `summary` returns : a dictionary from the number of guesses to the number of games **won** in that many, in order of guesses. Losses are left out. Four games won in 2, 4, 4 and lost, for example, give `{2: 1, 4: 2}`.

The chart comes in Step 6, where `print_stats` turns each count into a row of `#` characters.

The query is given :

```
"SELECT guesses, COUNT(*) FROM games WHERE won = 1 GROUP BY guesses
ORDER BY guesses"
```

Read it from the inside out. `WHERE won = 1` keeps only the won games. `GROUP BY guesses` makes one result row for each distinct value of `guesses`, and `COUNT(*)` counts the games in that group. `ORDER BY guesses` puts the rows in ascending order. Each result row is a `(guesses, count)` tuple. Your work is to run it and turn the rows into the dictionary, then add it to what `summary` returns :

```
    rows = connection.execute(
        "SELECT guesses, COUNT(*) FROM games WHERE won = 1 GROUP BY
guesses ORDER BY guesses"
    ).fetchall()
    distribution = ...
```

The Python session reads `stats.py` once, so start a new one after the change and record the same four games as in Step 3. `summary(connection)["distribution"]` is then `{2: 1, 4: 2}`, and the three totals are as before.

> ***Hint :*** *The slide Dictionary Construction with zip, in module 07, passes pairs of keys and values to* `dict` *.*

> ***Hint :*** `fetchall()` *returns a list of* `(guesses, count)` *tuples, and* `dict(...)` *turns a list of pairs straight into a dictionary.*

Solution on next page ↓

Solution

```
    rows = connection.execute(
        "SELECT guesses, COUNT(*) FROM games WHERE won = 1 GROUP BY
guesses ORDER BY guesses"
    ).fetchall()
    distribution = dict(rows)
```

Add `"distribution": distribution` to the dictionary that `summary` returns.

Step 5 : The current streak

The last statistic is the one players care about : how many games in a row have you won, up to now? Add the key `"streak"` to the dictionary : the number of wins since the most recent loss, `0` if the last game was lost or there are no games.

SQL can do this too, but the clearest solution reads the games newest first and counts in Python until it meets a loss. The query is given :

```
"SELECT won FROM games ORDER BY id DESC"
```

`id` grows with every game, so `ORDER BY id DESC` puts the newest game first. A cursor can be looped over directly, without `fetchall`, and every result row is a tuple, even with one column. This loop gives one `won` value per game; the parentheses unpack the one-item tuple :

```
for (game_won,) in connection.execute("SELECT won FROM games ORDER BY
id DESC"):
```

In a new Python session, record the four games of Step 3 : the last two were won, so the streak is `2`. Record one more lost game and it drops to `0`. An empty database has a streak of `0` too :

```
summary(connection)["streak"]                          # 2
record_game(connection, "crane", 6, False)
summary(connection)["streak"]                          # 0
summary(connect(":memory:"))["streak"]                 # 0
```

> ***Hint :*** *The slide break and continue Statements, in module 04, leaves a loop early*

with `break` *.*

Hint : *Start the streak at* `0` *, add one for every won game, and* `break` *at the first* `0` *.*

Solution on next page ↓

Solution

```
    streak = 0
    for (game_won,) in connection.execute("SELECT won FROM games ORDER
BY id DESC"):
        if not game_won:
            break
        streak += 1
```

Add `"streak": streak` to the dictionary. The finished function is in `solutions/10-database-access/wordle/stats.py`.

Step 6 : Use the database in the game

Everything is ready to replace the history file. In `wordle/game.py` :

- import `closing` from `contextlib`, and `connect` and `summary` from `wordle.stats`; `print_stats` needs all three
- remove `history_path`, `record_history` and the `csv` and `pathlib` imports that only they used
- in `play`, remove the two `record_history(...)` calls

`play` has returned a `bool` since Exercise 5, and nothing has used it yet. It is what the last argument of `record_game` needs, but `record_game` also needs the number of guesses, and `play` is the only place that knows it. Let `play` return both, as a tuple, and let the program that called `play` record the game :

```
def play(secret: str, words: list[str], max_guesses: int = 6) ->
tuple[int, bool]:
    """Play one game and return the guesses used and whether the secret
was found."""
    ...
        if guess == secret:
            print(f"Got it in {guess_count}!")
            return guess_count, True
    print(f"The word was {secret.upper()}.")
    return guess_count, False
```

In `wordle/__main__.py`, keep the secret in a variable, unpack what `play` returns and record the game. Add `from wordle.stats import connect, record_game` :

```
        secret = os.environ.get("WORDLE_SECRET", random.choice(words))
        guesses, won = play(secret, words)
        with closing(connect()) as connection, connection:
            record_game(connection, secret, guesses, won)
```

Add `from contextlib import closing` at the top. The `with` line has two parts, and each one has a job :

- `closing(connect())` closes the connection when the block ends. The `with` of a connection does **not** close it; it only manages the transaction, and a connection that is never closed is reported by Python 3.14 as an unclosed database. `closing` from the standard library calls `close()` for any object that has one.
- The second part, `connection`, is the transaction. When the block ends without an error the changes are committed, and when it ends with an error they are rolled back. This is the single place in the program where a game is committed : `record_game` only inserts, and the caller decides. The parts leave in reverse order, so the commit comes before the close.

A game you stop with Ctrl+C or Ctrl+D never reaches the `record_game` line, so it is not recorded. A game you give up with `quit` ends the loop and counts as lost.

Now rewrite `print_stats` so that the `stats` command reports the new numbers. It must print three lines : the totals and win rate with one decimal, the wins per guess count as `#` bars, and the streak :

```
def print_stats() -> None:
    """Print database totals, a guess chart and the current winning
streak."""
    with closing(connect()) as connection:
        totals = summary(connection)
    print(...)  # Games played: 4; won: 3; win rate: 75.0%.
    chart = ...
    print(...)  # Wins by guesses: 1: # 2: # 3: #
    print(...)  # Current streak: 1
```

Play four games with `WORDLE_SECRET=crane` : win in two (`slate` , `crane`), win in one (`crane`), give up (`quit`) and win in three (`slate` , `trace` , `crane`). Type `stats` at the first prompt of a fifth game :

```
Games played: 4; won: 3; win rate: 75.0%.
Wins by guesses: 1: # 2: # 3: #
Current streak: 1
```

Hint : *The slide String Operators, in module 02, lists* `*` *for repetition, Format Specifications shows a number with a fixed count of decimals, and Generator Expressions, in module 07, writes an expression inside* `join` *.*

Hint : *A chart for one guess count is* `f"{guesses}: {'#' * count}"` *, and a string can be multiplied by a number. Join the pieces with* `" ".join(...)` *over* `totals["distribution"].items()` *.*

Hint : *With no wins the chart is an empty string, which prints a confusing nothing.* `chart or 'none'` *prints* `none` *instead.*

Solution on next page ↓

Solution

```
def print_stats() -> None:
    """Print database totals, a guess chart and the current winning
streak."""
    with closing(connect()) as connection:
        totals = summary(connection)
    print(
        f"Games played: {totals['played']}; won: {totals['won']}; "
        f"win rate: {totals['win_percentage']:.1f}%."
    )
    chart = " ".join(
        f"{guesses}: {'#' * count}"
        for guesses, count in totals["distribution"].items()
    )
    print(f"Wins by guesses: {chart or 'none'}")
    print(f"Current streak: {totals['streak']}")
```

Delete `history.txt` if you want : the database has taken over. The finished project is in `solutions/10-database-access/` .

If time permits

- Import the `history.txt` of Exercise 9 into the database once, with a small script. Read the file with `csv.reader` and insert each row; the old file has no time, so store an empty `played_at` .
- Keep `wordle.db` in a fixed place. Build the path with `Path(__file__)` , the way you did for the history file in Exercise 9, so the statistics are the same wherever you start the game. There is no reference solution for these steps.

Exercise 11 : Python Classes

Look at `play` in `wordle/game.py`. The state of one game is spread over a handful of loose variables : `secret`, `words`, `history`, `guess_count`, `max_guesses`. Every new feature, the hint, the statistics and the next one you add, has to pass them around or reach into them, and nothing stops a function from putting a nonsense value in one of them. In this exercise you gather them in a class, `Game`, which owns its state and the rules about it. On the way you meet properties and the one question that decides when to use them.

Continue in your own `wordle` project, or start from `exercises/starters/11-python-classes/`. The starter is the finished Exercise 10.

Some steps give you the code, some a skeleton to complete, and one only the signatures and a few examples of what they should do. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards.

Step 1 : A Game with plain attributes

Add the class to `wordle/game.py`, above `play`. For now it only stores what a game needs :

```
class Game:
    """Keep one game's state."""

    def __init__(self, secret: str, words: list[str], max_guesses: int
= 6) -> None:
        self.secret = secret
        self.words = words
        self.history: list[tuple[str, list[str]]] = []
        self.max_guesses = max_guesses
```

`__init__` runs when you write `Game(...)`, and `self` is the new object. Every `self.name = ...` makes an attribute on that object. `secret` is the word to find, `words` is the list of words the game accepts as a guess, and `history` collects each

guess with its marks, which is the list that `candidates` in `wordle/solver.py` already expects. See *Creating Classes* and *Class Members*.

Start a Python session in the project folder and look at an object :

```
uv run python
```

```
from wordle.game import Game

game = Game("crane", ["crane", "slate"])
print(game.secret, game.max_guesses, game.history)
game.max_guesses = 3
print(game.max_guesses)
```

```
crane 6 []
3
```

An attribute is read and assigned like a variable. There is nothing else to write, and for now that is how it should be. See *Attributes Are Public*.

Step 2 : A method that plays a guess

A class is the data together with the functions that work on it. The first of those is the heart of the game : take a guess, check it, score it, remember it. Add a method to `Game` :

```
    def guess(self, word: str) -> list[str]:
        """Validate, score and record one guess, then return its
marks."""
        ...
```

The functions `check_guess` and `score_guess` already exist, and `game.py` already imports them. Use `self.words` , `self.secret` and `self.history` instead of arguments, because the object has them. An invalid word must raise `InvalidGuess` before anything is recorded.

Try it in the session :

```
game = Game("crane", ["crane", "slate"])
game.guess("slate")
game.history
game.guess("zzzzz")
```

```
['grey', 'grey', 'green', 'grey', 'green']
[('slate', ['grey', 'grey', 'green', 'grey', 'green'])]
wordle.scoring.InvalidGuess: That word is not in the word list.
```

The rejected guess left the history alone : `game.history` still holds only `slate` .

> ***Hint :*** *The slide Class Members shows a method that works on* `self` *, and List Functions and Methods, in module 03, has* `append` *.*

> ***Hint :*** *The method has three lines of real work :* `check_guess(word, self.words)` *, then* `marks = score_guess(self.secret, word)` *, then an* `append` *of the pair* `(word, marks)` *to* `self.history` *. Return* `marks` *.*

Solution on next page ↓

Solution

```
    def guess(self, word: str) -> list[str]:
        """Validate, score and record one guess, then return its
marks."""
        check_guess(word, self.words)
        marks = score_guess(self.secret, word)
        self.history.append((word, marks))
        return marks
```

`check_guess` comes first on purpose : when it raises, the exception leaves the method before the history changes.

Step 3 : Values that work themselves out

Callers will keep asking three things : has the player won, is the game over, how many guesses are left. All three can be worked out from `history` and `max_guesses`, so none of them needs to be stored. As methods they would be called `game.is_won()`. With `@property` above the `def` they are read as `game.is_won`, like an attribute that is always correct. See *A Computed Attribute*.

Add these three to `Game`, above `guess` :

```
    @property
    def is_won(self) -> bool:
        """Return whether the latest guess found the secret."""
        ...

    @property
    def is_over(self) -> bool:
        """Return whether the game was won or has no guesses left."""
        ...

    @property
    def guesses_left(self) -> int:
        """Return how many guesses remain."""
        ...
```

If you stored `is_won` as an attribute you would have to remember to update it in `guess`, and one forgotten line leaves it wrong. A computed value cannot go stale.

Now use `is_over` in `guess` as well : a finished game should refuse another guess. Put this at the top of the method :

```
        if self.is_over:
            raise ValueError("The game is over.")
```

Try the three, and see that they cannot be assigned to :

```
game = Game("crane", ["crane", "slate"], max_guesses=2)
print(game.is_won, game.is_over, game.guesses_left)
game.guess("slate")
print(game.is_won, game.is_over, game.guesses_left)
game.guess("crane")
print(game.is_won, game.is_over, game.guesses_left)
game.is_won = False
```

```
False False 2
['grey', 'grey', 'green', 'grey', 'green']
False False 1
['green', 'green', 'green', 'green', 'green']
True True 0
AttributeError: property 'is_won' of 'Game' object has no setter
```

Hint : *The slide A Computed Attribute shows* `@property` *on a method that returns a value worked out from other attributes.*

Hint : `is_won` *is true when there is at least one guess and the last one is the secret.* `self.history[-1][0]` *is the word of the last guess. An empty list has no last item, so check* `self.history` *first;* `bool(self.history and ...)` *returns a real* `bool` *.*

Hint : `guesses_left` *is* `self.max_guesses - len(self.history)` *.* `is_over` *is* `self.is_won` *or no guesses left; one property can use another. Test for* `<= 0` *rather than* `== 0` *, so that a game that somehow has more guesses than its limit counts as over too.*

Solution on next page ↓

Solution

```
    @property
    def is_won(self) -> bool:
        """Return whether the latest guess found the secret."""
        return bool(self.history and self.history[-1][0] ==
self.secret)

    @property
    def is_over(self) -> bool:
        """Return whether the game was won or has no guesses left."""
        return self.is_won or self.guesses_left <= 0

    @property
    def guesses_left(self) -> int:
        """Return how many guesses remain."""
        return self.max_guesses - len(self.history)
```

Step 4 : When a plain attribute is not enough

`max_guesses` is a plain attribute, so anybody can assign anything to it. Find out what that allows :

```
game = Game("crane", ["crane", "slate"])
game.max_guesses = 0
print(game.max_guesses, game.guesses_left, game.is_over)
game.max_guesses = -3
print(game.max_guesses, game.guesses_left, game.is_over)
```

```
0 0 True
-3 -3 True
```

Both games are over before they start, and the second one claims to have minus three guesses left. The class has no way to stop either. This is the situation the slide *When a Plain Attribute Is Not Enough* is about.

The fix is to turn `max_guesses` into a property with a setter, and to let the setter refuse values that are not between 1 and 10. Here are the signatures. Both methods are called `max_guesses`, and the second is decorated with `@max_guesses.setter`:

```
    @property
    def max_guesses(self) -> int:
        """Return the configured guess limit."""

    @max_guesses.setter
    def max_guesses(self, value: int) -> None:
        """Keep the guess limit between one and ten."""
```

The setter raises `ValueError("max_guesses must be between 1 and 10")` for any other value, and stores a valid one in `self._max_guesses`. The leading underscore says that the attribute is the property's own storage and not for outside use; see *The Rule of Thumb*.

In a new session, `10` is still accepted, but `0`, and a limit of `11` given to the constructor, now stop with the error:

```
game = Game("crane", ["crane", "slate"])
game.max_guesses = 10
print(game.max_guesses)
game.max_guesses = 0
Game("crane", ["crane", "slate"], max_guesses=11)
```

```
10
ValueError: max_guesses must be between 1 and 10
ValueError: max_guesses must be between 1 and 10
```

Make it work **without changing a line anywhere else**. `__init__` still says `self.max_guesses = max_guesses`, and `play` still reads `game.max_guesses`. That is the lesson : callers could not tell the plain attribute from the property, so nobody had to write `get_max_guesses` and `set_max_guesses` in advance. You start with the plain attribute, and you turn it into a property on the day it needs a rule.

> ***Hint :*** *The slide When a Plain Attribute Is Not Enough turns `length` into a property whose setter checks the value; `demo07 · validatingsetter` runs it.*

> ***Hint :*** *The getter returns `self._max_guesses`. The setter checks `1 <= value <= 10`, raises `ValueError` when that is false and assigns `self._max_guesses = value` otherwise.*

> ***Hint :*** *Assign to `self._max_guesses` in the setter, with the underscore. `self.max_guesses = value` inside the setter calls the setter again, forever, until Python stops with a `RecursionError`.*

Solution on next page ↓

Solution

```
    @property
    def max_guesses(self) -> int:
        """Return the configured guess limit."""
        return self._max_guesses

    @max_guesses.setter
    def max_guesses(self, value: int) -> None:
        """Keep the guess limit between one and ten."""
        if not 1 <= value <= 10:
            raise ValueError("max_guesses must be between 1 and 10")
        self._max_guesses = value
```

`Game("crane", words, max_guesses=11)` fails too, because `__init__` assigns through the property.

Step 5 : Draw the board and shrink play

A `Game` that can draw itself helps you debug it. Python asks an object for its text with `__str__`; `print(game)` and f-strings call it. Add one to `Game` that draws every scored guess on its own row with `render`.

```
    def __str__(self) -> str:
        """Draw every scored guess on its own row."""
        ...
```

```
game = Game("crane", ["crane", "slate"])
game.guess("slate")
game.guess("crane")
print(game)
```

```
['grey', 'grey', 'green', 'grey', 'green']
['green', 'green', 'green', 'green', 'green']
 S  L [A] T [E]
[C][R][A][N][E]
```

Now let `play` do less. Change it so that it creates a `Game` and loops until the game is over :

- create the game after loading the allowed list : `game = Game(secret, allowed_words, max_guesses)`
- loop with `while not game.is_over` , and remove the variables `history` and `guess_count`
- show the prompt number with `len(game.history) + 1` and the limit with `game.max_guesses`
- give `candidates(words, game.history)` the history for the hint
- in the `try` block, keep the `check_guess(guess, allowed_words)` line, and let `marks = game.guess(guess)` replace the scoring and recording below it
- after the loop, `game.is_won` decides the message : `Got it in N!` after a win, and `The word was ...` after a loss
- return `len(game.history), game.is_won` , the same two values as in Exercise 10, so that `__main__` still has what it needs for `record_game`

Play a game with `WORDLE_SECRET=crane` ; it must behave as before.

> ***Hint :*** *The slide `__str__` and `__repr__` shows `__str__` returning the text that `print` shows, and Generator Expressions, in module 07, writes an expression inside `join` .*

> ***Hint :*** *The loop body shrinks to : read the guess, handle the commands, `check_guess` and `marks = game.guess(guess)` inside `try` , update `used_letters` , print `render(guess, marks)` . The win test and the counter are gone, because the loop condition and `game.is_won` now know.*

> ***Hint :*** *A single `"\n".join(...)` over a generator expression is the whole of `__str__` . The expression is `render(word, marks)` for each `word, marks` pair in `self.history` , and `\n` is the newline that puts each row on its own line.*

Solution on next page ↓

Solution

```
    def __str__(self) -> str:
        """Draw every scored guess on its own row."""
        return "\n".join(render(word, marks) for word, marks in
self.history)
```

```
def play(secret: str, words: list[str], max_guesses: int = 6) ->
tuple[int, bool]:
    """Play one game and return the guesses used and whether the secret
was found."""
    allowed_words = load_allowed()
    game = Game(secret, allowed_words, max_guesses)
    used_letters = set()
    while not game.is_over:
        guess = input(f"Guess {len(game.history) +
1}/{game.max_guesses} : ").strip().lower()
        match guess:
            case "quit":
                break
            case "letters":
                print("Tried :", " ".join(sorted(used_letters)))
                continue
            case "hint":
                possible = candidates(words, game.history)
                print(f"{len(possible)} words left, try
{best_guess(possible).upper()}")
                continue
            case "stats":
                print_stats()
                continue
        try:
            check_guess(guess, allowed_words)
            marks = game.guess(guess)
        except InvalidGuess as error:
            print(error)
            continue
        used_letters.update(guess)
        print(render(guess, marks))
    if game.is_won:
        print(f"Got it in {len(game.history)}!")
    else:
```

```
        print(f"The word was {secret.upper()}.")
    return len(game.history), game.is_won
```

If time permits

- Add a `board` command to `play` that prints the game with `print(game)`.
- Write a `class Stats` that wraps the database connection of Exercise 10, with methods `record(secret, guesses, won)` and `summary()`, and let `__main__` use a `Stats` object instead of calling `connect()` and `record_game` itself. There is no reference solution for these steps.

Exercise 12 : Type Hinting APPENDIX

Your code has carried type hints since Exercise 2, and nobody has ever checked them. Python itself ignores them, so a wrong hint is only a comment that lies. In this exercise you hand the hints to `mypy`, a tool that reads the whole package without running it and reports where the types do not fit. It finds a bug that has been waiting in your solver all along, and it helps you describe the marks of a guess so precisely that a typo in them cannot slip through.

Continue in your own `wordle` project, or start from `exercises/starters/12-type-hinting/`. The starter is the finished Exercise 11; this exercise hands out nothing. Keep running commands beside `pyproject.toml`.

Some steps give you the code, some a list of changes to make, and one only a signature and what it should do. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards. This exercise is an appendix : do the steps in order.

Step 1 : Run mypy

`mypy` is a development tool, not something the game needs when it runs, so add it to the development dependencies, the way you added `ruff` in Exercise 1, and run it on the package :

```
uv add --dev mypy
uv run mypy wordle
```

```
wordle\game.py:65: error: Need type annotation for "used_letters"
(hint: "used_letters: set[<type>] = ...")  [var-annotated]
Found 1 error in 1 file (checked 8 source files)
```

Each message has a file, a line number, what is wrong and a code in brackets. The line number in your own project may differ. `used_letters = set()` is an empty set, and the checker cannot tell from that what it will hold. You know it holds letters. Tell it, with a hint on the variable, as you did for collections in *Typing Collections* :

```
    used_letters: set[str] = set()
```

Run `uv run mypy wordle` again :

```
Success: no issues found in 8 source files
```

See *Checking with mypy* for the same run on a smaller program.

Step 2 : Say exactly what a mark is

The marks that `score_guess` returns are declared as `list[str]` . That is true, but it is also true of `["gray", "blue", "yes"]` . The game only has three marks : `"green"` , `"yellow"` and `"grey"` . A type that lists exactly those can be checked.

In `wordle/scoring.py` , import `Literal` from `typing` , and give the three values a name below `WORD_LENGTH` :

```
from typing import Literal

type Mark = Literal["green", "yellow", "grey"]
```

A `Literal` type is the type of those values and no others : a `str` accepts `"gray"` as happily as `"grey"` , but `Mark` does not, so a misspelt mark becomes a message from `mypy` instead of a game that quietly shows the wrong colour. See *Closed Sets with Literal*. The `type` line defines a name for it, so the rest of the code can say `Mark` .

Now change one thing only : the return type of `score_guess` becomes `list[Mark]` . Run `mypy` and read what it finds :

```
wordle\scoring.py:46: error: Incompatible return value type (got
"list[str]", expected "list[Literal['green', 'yellow', 'grey']]")
[return-value]
wordle\game.py:53: error: Argument 1 to "append" of "list" has
incompatible type "tuple[str, list[Literal['green', 'yellow',
'grey']]]"; expected "tuple[str, list[str]]"  [arg-type]
wordle\game.py:54: error: Incompatible return value type (got
"list[Literal['green', 'yellow', 'grey']]", expected "list[str]")
[return-value]
Found 3 errors in 2 files (checked 8 source files)
```

`mypy` also prints a few lines starting with `note:` , which are left out here. A single change has shown you every place the idea of a mark runs through. Work through them :

- in `score_guess` , the list `marks` starts as `["grey"] * WORD_LENGTH` ; annotate it as `marks: list[Mark]`
- in `wordle/game.py` , import `Mark` from `wordle.scoring` , and use it in the annotation of `self.history` (`list[tuple[str, list[Mark]]]`) and in the return type of `Game.guess`
- the same two spots in `wordle/solver.py` (`consistent` and `candidates`) and the `marks` parameter of `render` in `wordle/display.py` are not flagged, but they describe marks too, so give them `Mark` as well

Run `uv run mypy wordle` until it reports `Success` . To see what you gained, misspell a mark in `score_guess` : change `"yellow"` into `"yelow"` , run `mypy` , and put it back. The first line of its report points at the assignment :

```
wordle\scoring.py:44: error: No overload variant of "__setitem__" of
"list" matches argument types "int", "str"  [call-overload]
```

The message is not friendly, but the file and line are right. Play a game as well : Python ignores annotations, so it plays as before.

> ***Hint :*** *The slide Closed Sets with Literal defines a* `Literal` *type with* `type` *and uses it in a signature.*

> ***Hint :*** *A* `type` *statement and a* `Literal` *need* `from typing import Literal` *and nothing else. Other modules import the alias like any name :* `from wordle.scoring import Mark` *.*

> ***Hint :*** `marks: list[Mark] = ["grey"] * WORD_LENGTH` *is an annotation on the first assignment. Without it, the checker infers* `list[str]` *for the list and complains when you return it as* `list[Mark]` *.*

Solution on next page ↓

Solution

In `wordle/scoring.py` :

```
from typing import Literal

WORD_LENGTH = 5

type Mark = Literal["green", "yellow", "grey"]


def score_guess(secret: str, guess: str) -> list[Mark]:
    """Return Wordle marks while counting each secret letter only
once."""
    marks: list[Mark] = ["grey"] * WORD_LENGTH
    ...
```

In `wordle/game.py` , `solver.py` and `display.py` , import `Mark` with `from wordle.scoring import Mark` (`game.py` already imports from that module, so add `Mark` to its import) and replace `list[str]` by `list[Mark]` where it describes marks :

```
        self.history: list[tuple[str, list[Mark]]] = []

    def guess(self, word: str) -> list[Mark]:

def consistent(word: str, guess: str, marks: list[Mark]) -> bool:

def candidates(words: list[str], history: list[tuple[str, list[Mark]]])
-> list[str]:

def render(guess: str, marks: list[Mark]) -> str:
```

Step 3 : A hint that may not exist

`best_guess` returns a word, and it returns it for any list you give it. Ask for a hint when no word is left and see what happens. This can only happen when the secret is a word the game accepts as a guess but never picks, so start the game with the environment variable `WORDLE_SECRET` set to `crone`, guess `crane`, and type `hint` :

```
ValueError: max() iterable argument is empty
```

The game stops with a traceback, and the type hints never warned about it, because `-> str` promised a word that cannot exist. The honest signature says that the answer may be missing. Python writes "a `str` or `None`" as `str | None`. See *Optional Values*.

Here is the new signature of `best_guess` in `wordle/solver.py` :

```
def best_guess(words: list[str]) -> str | None:
    """Choose the word with the most frequently occurring distinct
letters."""
```

Make it return `None` for an empty list. Then run `uv run mypy wordle`. The checker does not need to run the game to find the next problem : `play` calls `.upper()` on the result, and `None` has no `.upper()`.

```
wordle\game.py:76: error: Item "None" of "str | None" has no attribute
"upper"  [union-attr]
Found 1 error in 1 file (checked 8 source files)
```

That is the whole use of `| None` : the checker makes every caller deal with the case that there is no answer. Fix the `hint` command in `play` so that it prints `No candidate words remain.` when there is no suggestion, and the old message when there is one. Run `mypy` again until it reports `Success: no issues found in 8 source files`, then repeat the experiment with `crone` :

```
[C][R] A [N][E]
No candidate words remain.
```

The first line is the row for the guess `crane` ; the second is the answer to `hint` .

> ***Hint :*** *The slide Optional Values writes `| None` for a value that may be absent, and tests `is None` before it uses the value.*

> ***Hint :*** `if not words: return None` *at the top of* `best_guess` *is enough. The rest of the function stays as it is.*

> ***Hint :*** *In* `play` *, keep the result in a variable first :* `suggestion = best_guess(possible)` *. Then test* `if suggestion is None:` *and print the message,* `else` *print the old line using* `suggestion.upper()` *. After the test, the checker knows that* `suggestion` *is a* `str` *.*

Expected output :

```
Success: no issues found in 8 source files
```

Solution on next page ↓

Solution

```
def best_guess(words: list[str]) -> str | None:
    """Choose the word with the most frequently occurring distinct
letters."""
    if not words:
        return None
    counts = letter_counts(words)
    return max(words, key=lambda word: sum(counts[letter] for letter in
set(word)))
```

The `hint` command in `play` :

```
            case "hint":
                possible = candidates(words, game.history)
                suggestion = best_guess(possible)
                if suggestion is None:
                    print("No candidate words remain.")
                else:
                    print(f"{len(possible)} words left, try
{suggestion.upper()}")
                continue
```

The finished project is in `solutions/12-type-hinting/` .

If time permits

Make `mypy` stricter. Add a `[tool.mypy]` section with `strict = true` to `pyproject.toml` , run `uv run mypy wordle` , and fix what it reports. A project with this much annotation has little to fix : `summary` in `wordle/stats.py` returns a bare `dict` , and strict mode wants to know what is inside. There is no reference solution for this step.

Exercise 13 : Python Libraries APPENDIX

Your Wordle plays well, but it is still a game for one person, one sitting and one list of rules. The standard library has what it takes to change that without installing anything : `datetime` gives everybody the same word on the same day, `json` lets a game wait until you come back, `re` checks input with a pattern, and `sys.argv` turns options on and off from the command line. The third-party test runner `pytest` , which you add in Step 3, lets you check your own work. Each step uses one of them.

Continue in your own `wordle` project, or start from `exercises/starters/13-python-libraries/` . The starter is the finished Exercise 12; this exercise hands out nothing. Keep running commands beside `pyproject.toml` .

The steps are small and separate, one library each. Do Steps 1 to 3 in order, because they build up `main` and the options of the game. Step 4 (`json`) and Step 5 (`re`) can then be done in either order. Step 6 and Step 7 (`pytest`) come last, because they test what the earlier steps wrote; Step 3 sets `pytest` up.

Some steps give you the code, some a skeleton to complete, and some only a signature and the test it must pass. Where you write the code yourself, the solution is hidden until you open it. Try the step first; the solution is there for when you are stuck, or to compare afterwards. This exercise is an appendix : pick the libraries you want.

Step 1 : The daily word

Everybody who plays today should get the same word, and tomorrow a different one. No server is needed for that : count the days since a fixed starting date, and use the count to pick from the word list. See *The datetime Module* and *Date Arithmetic with timedelta*.

Try it in a Python session first, with a fixed date so that your result is the same as the one below :

```
uv run python
```

```
from datetime import date
from wordle.words import load_words

words = load_words()
today = date(2026, 10, 17)
day_number = (today - date(2026, 1, 1)).days
print(day_number, words[day_number % len(words)], len(words))
```

```
289 drill 935
```

Subtracting two dates gives a `timedelta`, and its `.days` is a whole number. The `%` operator is the remainder after division : after 935 days the count has passed the end of the word list, and the remainder starts again at the first word. Day 289 of 2026 is the word `drill`.

Now give the game an option. In `wordle/__main__.py` add `import sys` and `from datetime import date` at the top, and let `main` choose the secret itself :

```
        words = load_words()
        if "--daily" in sys.argv[1:]:
            day_number = (date.today() - date(2026, 1, 1)).days
            secret = words[day_number % len(words)]
        else:
            secret = os.environ.get("WORDLE_SECRET",
random.choice(words))
        print(
            "Guess the five-letter word. Type 'quit', 'letters', "
            "'hint' or 'stats'."
        )
        guesses, won = play(secret, words)
```

The lines that record the game stay as they are. `sys.argv` is the list of words typed on the command line; Step 2 looks at it properly. Start the game twice with `uv run python -m wordle --daily`, type `quit` each time and compare the word it reveals. It is the same word both times.

Step 2 : Options from the command line

`sys.argv` holds what was typed after the program name. Look at it :

```
uv run python -c "import sys; print(sys.argv)" --easy
```

```
['-c', '--easy']
```

The first item is the program, the rest are the options. Read *Command-Line Arguments with sys.argv*. Reading them by hand is fine for a few flags; `argparse` is the standard answer when there are many, and is named at the end of this exercise.

Give the game two more options : `--easy` gives the player ten guesses instead of six, and `--hard` switches on hard mode, which Step 3 builds. Rewrite the part of `main` from Step 1 so that it reads the options once :

```
        words = load_words()
        options = set(sys.argv[1:])
        if "--daily" in options:
            day_number = (date.today() - date(2026, 1, 1)).days
            secret = words[day_number % len(words)]
        else:
            secret = os.environ.get("WORDLE_SECRET",
random.choice(words))
        max_guesses = ...
        hard = ...
```

Fill in `max_guesses` (`10` with `--easy` , otherwise `6`) and `hard` (a `bool`). Then pass both to `play` , as `guesses, won = play(secret, words, max_guesses, hard)` . `play` has the `max_guesses` parameter already; give it the new one, which does nothing yet :

```
def play(secret: str, words: list[str], max_guesses: int = 6, hard:
bool = False) -> tuple[int, bool]:
```

Run `uv run python -m wordle --easy`. The prompt now reads `Guess 1/10`. A guess limit of 10 is allowed by the property you wrote in Exercise 11, and 11 would not be.

> ***Hint :*** *Exercise 10 gave a conditional expression for the win percentage, and the slide Logical, Membership, Identity Operators, in module 02, shows* `in` *giving* `True` *or* `False`*.*

> ***Hint :*** *A conditional expression fits on one line :* `10 if "--easy" in options else 6`*. A membership test is already a* `bool`*, so* `hard = "--hard" in options` *needs no* `if`*.*

Solution on next page ↓

Solution

```
        words = load_words()
        options = set(sys.argv[1:])
        if "--daily" in options:
            day_number = (date.today() - date(2026, 1, 1)).days
            secret = words[day_number % len(words)]
        else:
            secret = os.environ.get("WORDLE_SECRET",
random.choice(words))
        max_guesses = 10 if "--easy" in options else 6
        hard = "--hard" in options
        print(
            "Guess the five-letter word. Type 'quit', 'letters', "
            "'hint' or 'stats'."
        )
        guesses, won = play(secret, words, max_guesses, hard)
```

Step 3 : Hard mode

In hard mode, every guess must use what the earlier guesses revealed. Write the rule as a function in `wordle/game.py`, above `play` :

```
def follows_hints(guess: str, history: list[tuple[str, list[Mark]]]) -
> bool:
    """Return whether guess keeps green letters and includes yellow
ones."""
```

For every earlier guess and its marks :

- a letter that was green must be in the same place in the new guess
- a letter that was yellow must be somewhere in the new guess

This is the first function of this exercise that gets a test. `pytest` is a development tool, like `ruff`, so add it the same way; see *Testing with pytest* :

```
uv add --dev pytest
```

The tests live in a folder of their own, `tests/`, and import the `wordle` package from the project folder. Tell `pytest` to look there by adding this section to `pyproject.toml`:

```
[tool.pytest.ini_options]
pythonpath = ['.']
```

Create `tests/test_ex13.py` for the tests of this exercise, and put this first one in it:

```
"""The Exercise 13 tests written from scratch."""

from wordle.game import follows_hints
from wordle.scoring import Mark


def test_follows_hints_keeps_green_letters_and_uses_yellow_letters() ->
None:
    history: list[tuple[str, list[Mark]]] = [
        ("crane", ["green", "grey", "yellow", "grey", "grey"])
    ]
    assert follows_hints("caper", history)
    assert not follows_hints("paper", history)
    assert not follows_hints("clink", history)
```

The history says that after `crane`, the C was in the right place, and the A was in the word but not in the third place. `caper` obeys both. `paper` moves the C, and `clink` leaves out the A. Run `uv run pytest` and make the test pass. If you wrote a `follows_hints` in the last step of Exercise 5, compare it with this one.

Then use it in `play`. When `hard` is true and there is a history, a guess that does not follow the hints is refused with `That guess does not follow the revealed hints.` and does not count. Do the check after the word is known to be valid, because the function indexes into the

guess and a word of three letters would break it. Play with `uv run python -m wordle --hard`, with `WORDLE_SECRET=crane` set : guess `slate`, then `trick`.

> ***Hint :*** *The slide Zip Function, in module 07, pairs two sequences, and Iterating with enumerate, in module 04, adds the position.*

> ***Hint :*** *Loop over the history with `for previous, marks in history`. Then loop over the positions with `enumerate(zip(previous, marks, strict=True))`, which gives `position, (letter, mark)`. A `return False` the moment a rule breaks, and a `return True` after the loops.*

> ***Hint :*** *Each rule is one condition that says when it is broken :*
> *`mark == "green" and guess[position] != letter` for the green rule, and*
> *`mark == "yellow" and letter not in guess` for the yellow rule.*

Expected output :

The end of a passing run; timings vary.

```
tests\test_ex13.py .
[100%]

============================== 1 passed in 0.03s
==============================
```

Solution on next page ↓

Solution

```
def follows_hints(guess: str, history: list[tuple[str, list[Mark]]]) -
> bool:
    """Return whether guess keeps green letters and includes yellow
ones."""
    for previous, marks in history:
        for position, (letter, mark) in enumerate(zip(previous, marks,
strict=True)):
            if mark == "green" and guess[position] != letter:
                return False
            if mark == "yellow" and letter not in guess:
                return False
    return True
```

In `play`, inside the `try` block, after `check_guess`:

```
            check_guess(guess, allowed_words)
            if hard and game.history and not follows_hints(guess,
game.history):
                print("That guess does not follow the revealed hints.")
                continue
            marks = game.guess(guess)
```

Step 4 : Save and resume

A game should survive being interrupted. `json` turns lists, dictionaries, strings and numbers into text and back again; see *The json Module*. Save the secret, the guesses and the game's settings (the guess limit and hard mode) after every guess, and load them when the game starts. The settings matter : a game that was started with

`--easy` must still have ten guesses when it is resumed without the flag. Create `wordle/storage.py`. The path of the save file is given, built from the module's own location as in Exercise 9 :

```
"""Save and restore an unfinished game in JSON format."""

import json
from pathlib import Path

from wordle.scoring import Mark

SAVE_PATH = Path(__file__).resolve().parents[1] / "save.json"
```

Write three functions with these signatures :

```
def save_game(
    secret: str, history: list[tuple[str, list[Mark]]], max_guesses:
int, hard: bool
) -> None:
    """Write the secret, the scored guesses and the game's settings to
the save file."""


def load_saved_game() -> tuple[str, list[tuple[str, list[Mark]]], int,
bool] | None:
    """Read a saved game, returning None when no save exists."""


def clear_saved_game() -> None:
    """Remove the save after a game is complete."""
```

Save a dictionary with the keys `"secret"`, `"history"`, `"max_guesses"` and `"hard"`, written with `json.dumps(data, indent=2)` so that you can read it. JSON has no tuples : the `(word, marks)` pairs are written as lists, and `load_saved_game` must turn them back into tuples, and return the secret, the history, the guess limit and the hard flag, in that order. See *Reading and Writing JSON Files* and *JSON and Python Types*.

Then connect the module to the game. This part is given.

In `wordle/game.py`, import the functions with `from wordle.storage import clear_saved_game, save_game`, and let `play` start from a history:

```
def play(secret: str, words: list[str], max_guesses: int = 6, hard:
bool = False,
         history: list[tuple[str, list[Mark]]] | None = None) ->
tuple[int, bool]:
```

```
    game = Game(secret, allowed_words, max_guesses)
    if history:
        game.history.extend(history)
        print(game)
    used_letters: set[str] = set()
    used_letters.update(letter for word, _ in game.history for letter
in word)
```

The `print(game)` shows the restored board before the first prompt, so that a player who comes back sees where the game stood; it uses the `__str__` of Exercise 11. Save after each printed row, with `save_game(game.secret, game.history, game.max_guesses, hard)`, and call `clear_saved_game()` once the loop has ended, so that a finished game leaves no save behind.

In `wordle/__main__.py`, add `from wordle.storage import load_saved_game`, and put the saved game first, before the options decide on a new one :

```
        words = load_words()
        options = set(sys.argv[1:])
        saved = load_saved_game()
        if saved is not None:
            secret, history, max_guesses, hard = saved
            print("Resuming your saved game.")
        else:
            history = []
            if "--daily" in options:
                day_number = (date.today() - date(2026, 1, 1)).days
                secret = words[day_number % len(words)]
            else:
                secret = os.environ.get("WORDLE_SECRET",
random.choice(words))
            max_guesses = 10 if "--easy" in options else 6
            hard = "--hard" in options
```

The limit and the hard flag now come from the save when there is one, and from the options only for a new game. Then call `guesses, won = play(secret, words, max_guesses, hard, history)`.

Play with `WORDLE_SECRET=crane`, guess `slate` and then stop the game with Ctrl+C. Open `save.json` :

```
{
  "secret": "crane",
  "history": [
    [
      "slate",
      [
        "grey",
        "grey",
        "green",
        "grey",
        "green"
      ]
    ]
  ],
  "max_guesses": 6,
  "hard": false
}
```

Start the game again, without any option. It announces the saved game, draws the board so far, continues at guess 2 with the same settings as before, and removes the save file when the game is finished :

```
Resuming your saved game.
Guess the five-letter word. Type 'quit', 'letters', 'hint' or 'stats'.
 S  L [A] T [E]
Guess 2/6 :
```

Try the same with `--easy`, and stop after a guess : the resumed game still says `Guess 2/10`.

A game that ends with `quit` is over and is cleared too. Only an interruption leaves a save behind.

> ***Hint :*** *The slide The json Module turns a dictionary into text with* `json.dumps` *and back with* `json.loads` *, and Renaming and Deleting Files, in module 09, removes a file with* `unlink` *.*

> ***Hint :***
> `SAVE_PATH.write_text(json.dumps(data, indent=2), encoding="utf-8")` *writes the file.* `json.loads(SAVE_PATH.read_text(encoding="utf-8"))` *reads it back as dictionaries and lists.*

> ***Hint :*** *Return* `None` *when* `SAVE_PATH.exists()` *is false.*
> `SAVE_PATH.unlink(missing_ok=True)` *removes a file, and does not complain if there is none.*

Solution on next page ↓

Solution

```
def save_game(
    secret: str, history: list[tuple[str, list[Mark]]], max_guesses:
int, hard: bool
) -> None:
    """Write the secret, the scored guesses and the game's settings to
the save file."""
    data = {"secret": secret, "history": history, "max_guesses":
max_guesses, "hard": hard}
    SAVE_PATH.write_text(json.dumps(data, indent=2), encoding="utf-8")


def load_saved_game() -> tuple[str, list[tuple[str, list[Mark]]], int,
bool] | None:
    """Read a saved game, returning None when no save exists."""
    if not SAVE_PATH.exists():
        return None
    data = json.loads(SAVE_PATH.read_text(encoding="utf-8"))
    history = [(word, marks) for word, marks in data["history"]]
    return data["secret"], history, data["max_guesses"], data["hard"]


def clear_saved_game() -> None:
    """Remove the save after a game is complete."""
    SAVE_PATH.unlink(missing_ok=True)
```

Step 5 : Check input with a pattern

`check_guess` uses `guess.isalpha()` to see that a guess is made of letters. It accepts more than the game can use, such as letters with an accent. Try it :

```
from wordle.scoring import check_guess
from wordle.words import load_allowed

check_guess("éclat", load_allowed())
```

```
wordle.scoring.InvalidGuess: That word is not in the word list.
```

`éclat` passed the letters test and was only stopped by the word list. A regular expression says exactly what is allowed : five characters, each one of `a` to `z`. Replace the test in `check_guess` with `re.fullmatch`, which succeeds only if the whole string matches the pattern. Add `import re` to `wordle/scoring.py`, and keep the check on the length before it :

```
    if not re.fullmatch(..., guess):
        raise InvalidGuess("A guess must contain letters only.")
```

See *Regular Expressions with re* and *Matching and Searching*. Run the session again; the same word is now refused for the right reason :

```
wordle.scoring.InvalidGuess: A guess must contain letters only.
```

Run `uv run pytest` ; the hard-mode test of Step 3 still passes.

> ***Hint :*** *The slide Matching and Searching uses `re.fullmatch` with a character class, `[a-z]+` .*

> ***Hint :*** *`[a-z]` is one lower-case letter and `{5}` repeats the previous item five times. Write the pattern as a raw string, `r"[a-z]{5}"` , as you would a path, so that backslashes in later patterns mean what they say.*

Solution on next page ↓

Solution

```
import re
...
    if not re.fullmatch(r"[a-z]{5}", guess):
        raise InvalidGuess("A guess must contain letters only.")
```

Step 6 : Write your own tests

In Step 3 the test was given. Tests of this kind are small : a function whose name starts with `test_` , and a plain `assert` . See *Testing with pytest*. You wrote this exercise's functions yourself, so write the tests too. Add two to `tests/test_ex13.py` , which Step 3 created. Their imports go at the top of the file, next to the ones that are there, and the tests go at the bottom :

```
import pytest
from wordle.scoring import InvalidGuess, check_guess
from wordle.solver import best_guess
```

```
def test_best_guess_returns_none_for_no_candidates() -> None:
    ...


def test_check_guess_rejects_capital_letters() -> None:
    ...
```

The first is one `assert` : with no candidates, `best_guess` returns `None` (the `str | None` of Exercise 12). Compare with `is None` . The second needs the test to expect an exception, which is what `pytest.raises` is for :

```
    with pytest.raises(InvalidGuess, match="letters only"):
        ...
```

Anything raised inside the `with` block must be an `InvalidGuess` whose message contains `letters only` , or the test fails. The pattern of Step 5 refuses capital letters. See *Exceptions and Parametrized Tests*. Read a failure before you move on :

change an expected value in one test on purpose, run `uv run pytest tests/test_ex13.py`, read what pytest shows next to the failing `assert`, and change it back. See *Reading a Test Failure*.

> ***Hint :*** *The slide Testing with pytest shows a test that is one `assert`, and Exceptions and Parametrized Tests puts the call that must fail inside `with pytest.raises(...)`.*

> ***Hint :*** *`assert best_guess([]) is None`. The second test calls `check_guess("CRANE", ["crane"])` inside the `with` block; the length is fine, so the pattern is what fails.*

Solution on next page ↓

Solution

```
def test_best_guess_returns_none_for_no_candidates() -> None:
    assert best_guess([]) is None


def test_check_guess_rejects_capital_letters() -> None:
    with pytest.raises(InvalidGuess, match="letters only"):
        check_guess("CRANE", ["crane"])
```

Step 7 : One test, several cases

Testing `clean_words` from Exercise 9 on several inputs means repeating a test function, or looping inside it, where the first failure hides the rest. `pytest.mark.parametrize` runs one test function once per value and reports each separately. Add this to `tests/test_ex13.py`, with `from wordle.words import clean_words` at the top :

```
@pytest.mark.parametrize("line", ["crane", "slate", "raise"])
def test_clean_words_keeps_lowercase_five_letter_words(line: str) ->
None:
    assert clean_words([line, line.upper()]) == [line]
```

The decorator names the parameter and lists the values; pytest calls the function with each. The test checks that cleaning a word and its capitalised copy leaves exactly one lower-case word. Run it with `-v` to see each case :

```
uv run pytest tests/test_ex13.py -v
```

```
tests/test_ex13.py::test_follows_hints_keeps_green_letters_and_uses_ye
llow_letters PASSED [ 16%]
tests/test_ex13.py::test_best_guess_returns_none_for_no_candidates
PASSED [ 33%]
tests/test_ex13.py::test_check_guess_rejects_capital_letters PASSED
[ 50%]
tests/test_ex13.py::test_clean_words_keeps_lowercase_five_letter_words[
crane] PASSED [ 66%]
tests/test_ex13.py::test_clean_words_keeps_lowercase_five_letter_words[
slate] PASSED [ 83%]
tests/test_ex13.py::test_clean_words_keeps_lowercase_five_letter_words[
raise] PASSED [100%]
```

Three cases, three lines. Finish with `uv run pytest` for the whole project, and `uv run mypy wordle` to check that the new modules are typed too.

If time permits

- Replace the hand-made option parsing in `main` with `argparse`, which also provides `--help` for free. Look it up in the standard library documentation. There is no reference solution for this step.
- Use `datetime` once more : store the date with the daily word in `save.json`, so that a saved daily game from yesterday is not resumed today.